# Antiphase: Recovery-Safe GPU Sharing Between Neural Receivers and AI Inference in AI-RAN

ANONYMOUS AUTHOR(S)

AI-RAN servers run the uplink physical layer and AI inference on the same GPUs, and neural receivers are the next physical-layer workload on these GPUs. This paper measures what a neural receiver is worth on such a server and what AI inference costs it. With the same number of LDPC iterations in both receivers, the real-time model of the public NVlabs receivers helps only when the channels of the paired users are correlated. The larger model decodes 15-67% of the transport blocks (TBs) that the cuPHY receiver fails on every channel we test, and it takes 2.5× the uplink period. Thus, the model that helps can run only for failed TBs, as a recovery path. We find that AI inference costs the recoveries of this path by a mechanism that deadline checks miss: a failed TB cannot wait, and a neural receiver that AI slows past its slack (1.2 ms at the median here) holds its GPU for three uplink slots, not two. A loss model of this path predicts the lost recoveries of 672 measured runs within 0.82 percentage points on average, and 24 values of six settings that had never been run, computed before the runs, within 0.34 points. The model points to a rule, which we build as Antiphase: the AI of a GPU stops while the neural receiver of that GPU runs. On four A100 GPUs with 16 cells, Antiphase serves 87% of the AI of a clairvoyant controller that we ran with the failures of every slot known in advance, 1.9-6.4× the existing mechanisms that keep the recoveries, and adds 0.04-0.06 ms to the 99.9th-percentile latency of the layer-1 decode, 2.8-21× less than the baselines. With a rate controller in the loop, the recovery path raises the uplink goodput of two-user cells by 11-40% when the channels of the users are correlated. Antiphase loses at most 0.5% of this goodput, as the mean of each of 42 settings, with one configuration: seven SNRs, three channels, an SNR that changes in steps and continuously, and cells with different channels on one server. No fixed or low-priority GPU share does: a share that serves more AI than Antiphase pays up to 8.2% of the goodput, and with a high neural receiver demand a 10% share already loses 0.8-0.9%.

CCS Concepts: • **Computer systems organization** → **Real-time systems**; • **Networks** → *Network resources allocation.*

Additional Key Words and Phrases: AI-RAN, GPU sharing, neural receiver, loss model, MPS, admission control

**ACM Reference Format:**
Anonymous Author(s). 2027. Antiphase: Recovery-Safe GPU Sharing Between Neural Receivers and AI Inference in AI-RAN. In *Proceedings of ACM SIGMETRICS 2027 (SIGMETRICS '27).* ACM, New York, NY, USA, 43 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

## 1 Introduction

Radio access networks (RANs) increasingly run baseband processing on GPUs, and AI-RAN platforms run radio physical-layer (PHY) processing and AI inference on the same GPUs [5, 20, 25, 28]. To use the GPU time that radio processing leaves idle, these platforms add AI services, such as large language model (LLM) inference, next to deadline-critical PHY tasks [19, 27]. Neural receivers are a PHY workload that these platforms now run on the GPU [20, 32]. A neural receiver replaces channel estimation, equalization, and demapping with a learned model and decodes transport blocks (TBs) that a conventional receiver fails [9, 17, 32]. Each TB that it decodes avoids a retransmission and frees an uplink slot.

The challenge stems from two facts that we measure in this paper. First, the neural receiver that helps is expensive. With the same number of LDPC iterations in both receivers, the real-time model of the public NVlabs receivers [10, 32] decodes more TBs than the cuPHY receiver only when the channels of the paired users are correlated. The larger model decodes more TBs on every channel we test, and it takes 5.3 ms per slot against an uplink period of 2.5 ms. Thus, it cannot run for every slot. Second, the larger model shares the GPU with the conventional receivers of all cells and with AI inference, and the existing GPU controls do not protect its recoveries. NVIDIA Multi-Process

Table 1. What the closest public designs run on the shared GPU.

| System | Radio work on the GPU | How AI shares the GPU |
|---|---|---|
| YinYangRAN, CloudRIC, Concordia | LDPC decoding or queued PHY jobs | GPU share or reserved capacity, changed at intervals of one second or more |
| DARIS, REEF, XSched, Orion, SMEC, nvtaskset | None; generic GPU tasks | Kernel, context, MPS, or partition priority without radio deadlines |
| ARCHES, OCUDU, neural receivers | Neural channel estimator or neural receiver | No AI workload on the same GPU |
| Antiphase | Conventional receiver for every TB and a neural receiver for failed TBs | Low-priority AI pieces that stop when the neural receiver of the GPU starts |

Service (MPS) caps the GPU threads of a process, but the cap is fixed when the process starts [21]. A cap that protects the radio work at full load leaves the GPU idle when the load drops. A low MPS priority for AI keeps the layer-1 deadline, and almost every neural receiver run still ends before its deadline, but 6.3% of the recoveries are lost at full load. We find that 83-99% of the lost recoveries are failed TBs that found no free neural receiver. A failed TB cannot wait. A neural receiver that runs alone is free again for the failed TBs of the second uplink slot after its own, and one that AI slows from 6.3 to 7.9 ms is free one slot later.

Existing systems cover one side of this problem. As Table 1 summarizes, vRAN and AI-RAN resource managers [5, 13, 19, 25–28] share GPUs between radio processing and other workloads by a GPU share or by reserved capacity, and their radio work is LDPC decoding or the conventional PHY. GPU scheduling systems [6, 7, 15, 29, 30, 35] prioritize or preempt GPU kernels and contexts without radio deadlines. Neural receiver studies and PHY platforms [9, 17, 22, 24, 32] design the receiver or switch between PHY models, and they do not share the GPU with an AI workload. However, none of them runs a neural receiver next to AI inference on one GPU, and none explains how AI inference costs the recoveries of such a receiver.

In this paper, we measure the neural receiver as a workload on a shared AI-RAN GPU, model what AI inference costs it, and present Antiphase, the sharing rule that the model points to. Antiphase runs AI in antiphase with the neural receiver: the AI of a GPU stops when the neural receiver of that GPU starts and resumes when it ends. We make three contributions.

- **A measurement.** We compare the public neural receivers with a production receiver at equal LDPC effort. The real-time model helps only when the channels of the paired users are correlated. The larger model decodes 15-67% of the TBs that cuPHY fails, and it is too slow to run for every slot. As a recovery path for failed TBs, it raises the uplink goodput of two-user cells by 5.6% at a fixed MCS, and by 11-40% on a high-correlation channel and 4-9% on a low-correlation channel when an outer-loop rate controller picks the MCS.
- **A finding and a model.** AI inference costs recoveries by a path that deadline checks miss. A neural receiver that AI slows by more than 1.2 ms still meets its deadline, but it holds its GPU for three uplink slots, not two, and the failed TBs of the extra slot find no free neural receiver. This path accounts for 83-99% of the lost recoveries. A baseline that admits AI whenever the neural receiver still meets its own deadline keeps every run inside the deadline and loses 6.2% of the recoveries. We treat the neural receivers as the servers of a loss system and compute the share of failed TBs that find none of them free from the run lengths of the neural receiver. The closed form predicts the loss of 672 measured runs with a correlation of 0.994 and a mean absolute error of 0.82 percentage points. We measured 260 of these runs after the model was final. For six settings that had never been run we committed the predicted loss of four policies before the runs; the 24 values were within

0.34 points on average. A controller that knows the failures of every slot in advance, which we ran, gives a reference for the AI that can be served while the recoveries are kept.

- **A design.** The model says what to protect: the run length of the neural receiver, not its deadline. Antiphase admits AI in stoppable pieces at low priority and stops the AI of a GPU when the neural receiver of that GPU starts. Its recovery path sends a failed TB to the neural receiver only when the number of failed code blocks predicts a successful decode, under a recovery deadline later than the layer-1 deadline, and the AI unit sizes that may run next to the conventional receivers follow from measured time bounds.

We evaluate Antiphase on one to four A100 GPUs with 4-48 cells, the cuPHY receiver, the public NVlabs neural receivers, and four AI models. An AI-RAN server shares its GPUs to use the time that the radio leaves, and the radio must keep its performance. Therefore, we ask of every policy first whether it keeps the radio and then how much of the GPU it uses. The main results are as follows.

- **Antiphase keeps the radio.** With one configuration, Antiphase keeps the radio in every setting we measure. With an outer-loop rate controller, the two-user cells lose at most 0.5% of their goodput against the server without AI in all 42 settings, and at most 0.3% in 41 of them (means per setting; the largest is 0.35% over its 33 seeds, with a 95% interval of 0.25-0.44%): seven SNRs, three channels, an SNR that changes in steps and continuously, and cells with different channels on one server. In steady state Antiphase adds 0.04-0.06 ms to the 99.9th-percentile latency of the layer-1 decode and 0.07-0.09 ms to the run of the neural receiver, and after the start of a run four to eight of a million TBs pass the layer-1 deadline, against two without AI (36 runs of 100 s). It loses 0.0-0.1% of the recoveries.
- **A GPU share does not.** A fixed share and a low-priority share run AI whatever the radio does, and no cap keeps the radio in every setting. The 10% share loses up to 1.1% of the goodput, and the 30% share with low-priority AI loses up to 2.3%. A higher cap serves more AI and loses more goodput in every setting, and no cap serves at least the AI of Antiphase at no more loss. The shares add 0.14-0.87 ms to the layer-1 tail, 2.8-21× what Antiphase adds, and with one GPU 21-46 of 0.8 million TBs pass the layer-1 deadline, against none with Antiphase and none without AI. A share chosen every second by a load estimator loses 2.2-4.7% of the recoveries when the load changes.
- **Antiphase uses more of the GPU.** Antiphase serves 34.1k-44.7k tokens/s within the 200 ms limit under three load patterns. This is 6.4-7.5× a fixed 10% share, 1.9-2.0× a 30% share with low-priority AI, and 1.2-1.9× a share chosen by a load estimator with low-priority AI, the strongest combination of existing mechanisms that we built. Against a clairvoyant controller that we ran, which knows the failures of every slot in advance, Antiphase serves 87% at full load and 76-95% in six settings, and the mechanisms that keep the recoveries serve 14-46%. Against an estimate for an offline schedule, which is not run, Antiphase serves 70%. The 99th-percentile latency of its AI requests is 175-187 ms, inside the limit.
- **More AI than Antiphase is paid by the radio.** A low-priority share with a cap of 70% or without a cap serves 1.1-1.4× the AI of Antiphase under the three load patterns and up to 4.2× under rate control. It pays with 2.7-6.3% of the recoveries, up to 8.2% of the goodput under rate control, and up to 0.9 ms of the layer-1 tail, and the price of a cap changes with the load and the channel.
- **The recovery path is worth protecting.** It adds 5.6% to the uplink goodput of two-user cells at a fixed MCS. When an outer-loop rate controller picks the MCS, it adds 11-40% on a high-correlation channel and 4-9% on a low-correlation channel.

Table 2. TBs decoded at 20 LDPC iterations in both receivers. The percentage is the share of the TBs cuPHY fails that the neural receiver decodes; the time is the GPU time of the model alone.

| Channel of the two users | TBs | cuPHY | Real-time model (1.35 ms) | Larger model (4.29 ms) |
|---|---|---|---|---|
| Low correlation, 2-3 dB | 600 | 515 | 484 (2%) | 572 (67%) |
| Urban micro, 1-6 dB | 600 | 425 | 420 (2%) | 451 (15%) |
| Medium correlation, 8-18 dB | 600 | 321 | 372 (21%) | not measured |
| High correlation, 10-16 dB | 1,400 | 946 | 1,099 (38%) | 1,159 (51%) |

## 2 Background and Motivation

### 2.1 AI-RAN on shared GPUs

An AI-RAN server decodes the uplink of many cells on GPUs and serves AI requests with the GPU time that is left [5, 20].

**Uplink Timeline.** With the TDD pattern DDDSU at 30 kHz subcarrier spacing, every cell has one uplink slot per 2.5 ms. The received samples of a slot are ready 0.5 ms after the slot starts, and the layer-1 result is due 4.5 ms after the slot starts, which leaves 4.0 ms for decoding. A TB that fails is retransmitted three uplink slots later, 7.5 ms after the original slot. The MAC fixes an uplink grant about three slots before it goes on air (the Aerial L2 adapter uses a slot advance of three [20]). With the result at 4.5 ms, the grant goes out 6.0-7.0 ms after the slot starts with a scheduling offset of three slots to one, and the PUSCH preparation time is 12 symbols [4].

**GPU Sharing.** MPS runs the processes of one GPU concurrently and caps the active threads of each process [21]. YinYangRAN changes the share between LDPC decoding and other workloads with a learned reliability estimator at intervals of at least one second, because an MPS reconfiguration takes 0.27 s [25], and CloudRIC pools GPUs for LDPC decoding across distributed units [26]. The share of an MPS client cannot change after the client starts. Therefore, a system that follows the load must restart the client or keep one loaded client per share.

### 2.2 When does a neural receiver help?

We compare the public NVlabs neural receivers [9, 10, 32] with the cuPHY PUSCH receiver [20] on the same received slots. Two users transmit on the same 273 PRBs with 16QAM, and the base station has four antennas. The slots come from the NVlabs link simulation [18]; cuPHY decodes them with MMSE equalization and time interpolation.

**Same LDPC Effort.** A receiver comparison is valid only when both receivers run the same number of LDPC iterations. cuPHY uses 10 iterations for the TB sizes in our setup, and the evaluation code of the neural receivers uses 20. With 10 against 20 iterations, the real-time model appears to decode 25-35% of the TBs that cuPHY fails on the low-correlation channel. With 20 iterations on both sides, the figure is 2%. Raising cuPHY from 10 to 20 iterations alone recovers 152 of 158 failed single-user TBs, at 15-19% more receiver time for TBs of 13-20 code blocks. Therefore, every result in this paper uses 20 iterations in both receivers. Table 2 shows the comparison.

**The Model That Helps Is Too Slow for Every Slot.** The real-time model decodes more TBs than cuPHY only when the channels of the two users are correlated. The larger model decodes more TBs on all three channels and loses none of the TBs that cuPHY decodes on two of them. It takes 5.3 ms per slot alone and 6.3 ms next to the cell workload, against an uplink period of 2.5 ms; cuPHY takes 0.8-0.9 ms. When we send every slot of four cells to the larger model, 61% of the slots find no free GPU in time and 89.4% of the TBs are decoded. When we send only the TBs that cuPHY fails, the neural receiver runs for 25% of the slots, the decode result is the same offline, and 94.5-95.1% of the TBs are decoded in the timed system.

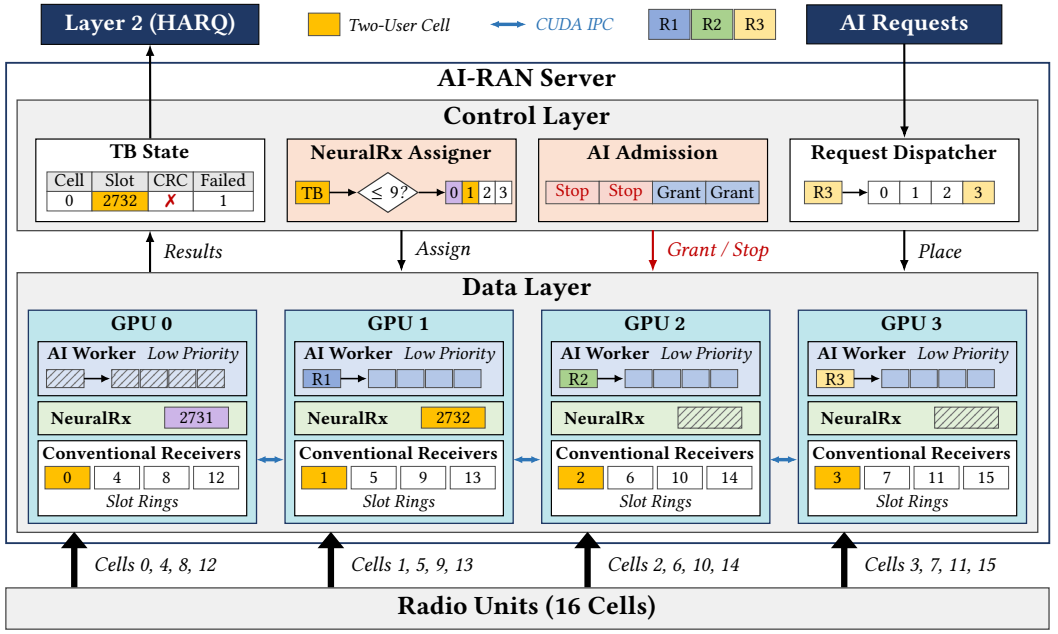



Fig. 1. Overall architecture of Antiphase. Every GPU runs the conventional receivers of four cells, one neural receiver, and one AI worker. The contents of the boxes show the instant of Figures 2 and 4: two neural receivers are busy, and the AI workers of GPU 1-3 hold a request.

**Failed Code Blocks Predict the Outcome.** The conventional receiver reports how many code blocks of a TB failed. On the urban micro channel, 87% of the failed TBs have every code block failed and the neural receiver decodes 2% of them. It decodes 93% of the TBs in slots with at most nine failed code blocks. On the low-correlation channel, it decodes 97% of the failed TBs in slots with one to three failed code blocks and 30% in slots with ten or more.

This motivates Antiphase's design: run the larger model only for failed TBs that it is likely to decode, give that run a deadline later than the layer-1 deadline, and admit AI only where it does not cost recoveries.

## 3 Antiphase Design

### 3.1 Overall architecture

Figure 1 shows the architecture of Antiphase. A server runs one controller and, on every GPU, the conventional receivers of the cells of that GPU, one neural receiver, and one AI worker under MPS. All processes read one shared state in memory and one clock.

**Control Layer.** The controller runs four functions in one loop of 15-57 $\mu$s. *TB State* holds one row per cell and slot with the result of the conventional receiver, the number of failed code blocks, and the state of the recovery. *NeuralRx Assigner* gives a failed TB to a free neural receiver (Section 3.2). *AI Admission* grants AI pieces to the AI worker of every GPU and stops them (Section 5). *Request Dispatcher* places every AI request on the GPU that finishes it first and rejects a request that cannot meet its time limit.

**Data Layer.** A conventional receiver process per cell decodes the TBs of its cell with cuPHY every 2.5 ms and keeps the samples of the last slots in a *slot ring* in GPU memory. One neural receiver process per GPU decodes one failed TB at a time. It maps the slot rings of all cells over CUDA IPC

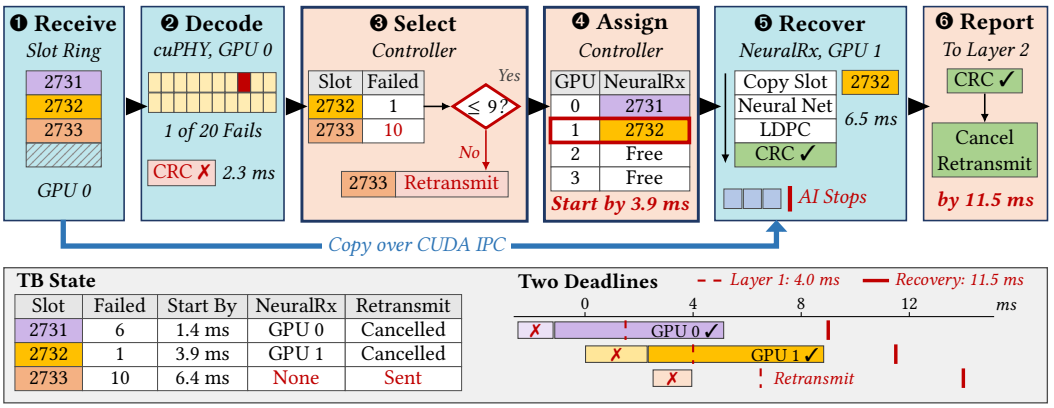



Fig. 2. Recovery path as six steps, with three consecutive failed TBs of one cell in a measured run (cell 0, slots 2731-2733). Blue steps run on a GPU and peach steps in the controller. Every TB keeps its color. Bottom: *TB State* after the six steps and the two deadlines of every TB; times count from the arrival of the samples of slot 2732.

and can take a TB of any cell on any GPU. One AI worker per GPU runs AI inference in pieces at a low MPS priority.

### 3.2 Recovery path

Figure 2 follows three consecutive failed TBs of one cell in a measured run through the six steps of the recovery path. ❶ *Receive.* The samples of a slot arrive in the slot ring of the cell on GPU 0. ❷ *Decode.* cuPHY decodes the slot and writes the CRC result and the number of failed code blocks to *TB State*: the CRC of slot 2732 fails 2.3 ms after the samples arrive, with one failed code block of 20. ❸ *Select. NeuralRx Assigner* keeps a failed TB as a candidate only if its slot has at most nine failed code blocks. Slot 2733 has ten and is retransmitted. ❹ *Assign.* The assigner gives the candidate with the earliest latest start time to a free neural receiver; among the free neural receivers, it takes one on a GPU without a running AI piece first. The neural receiver of GPU 0 is still running the TB of slot 2731, and the TB of slot 2732 goes to GPU 1. ❺ *Recover.* The neural receiver of GPU 1 copies the slot from the ring on GPU 0 over CUDA IPC (blue arrow) and runs the neural network, LDPC decoding, and the CRC on its own GPU, from 2.3 to 8.8 ms. The AI of GPU 1 stops when this run starts (Section 5). ❻ *Report.* The result arrives before the recovery deadline and cancels the retransmission. *TB State* at the bottom of the figure holds one row per TB with the values that the steps write.

**Two Deadlines.** Every TB has a layer-1 deadline of 4.0 ms after its samples arrive, which the conventional receiver must meet. A TB that fails has a *recovery deadline*: if the neural receiver decodes it before this deadline, the retransmission is cancelled. The recovery deadline is a parameter. Keeping the retransmission slot of the conventional path needs the result within 5.0 ms, and the larger model does not fit into it. Antiphase uses 11.5 ms. The decision then falls 12.0 ms after the slot starts, and with a MAC lead of three slots the retransmission of a TB that is not recovered takes the uplink slot at 15.0 ms, three uplink slots (7.5 ms) after the conventional one. For that slot the decision may be as late as 12.5-13.0 ms, so 11.5 ms is 0.5-1.0 ms early. NR permits this without a change at the UE: uplink HARQ is asynchronous, the gNB schedules every retransmission with a grant [3], and the UE keeps the data of a HARQ process until a grant with a toggled new-data indicator arrives [2]. The bottom of Figure 2 shows both deadlines for the three TBs: cuPHY ends

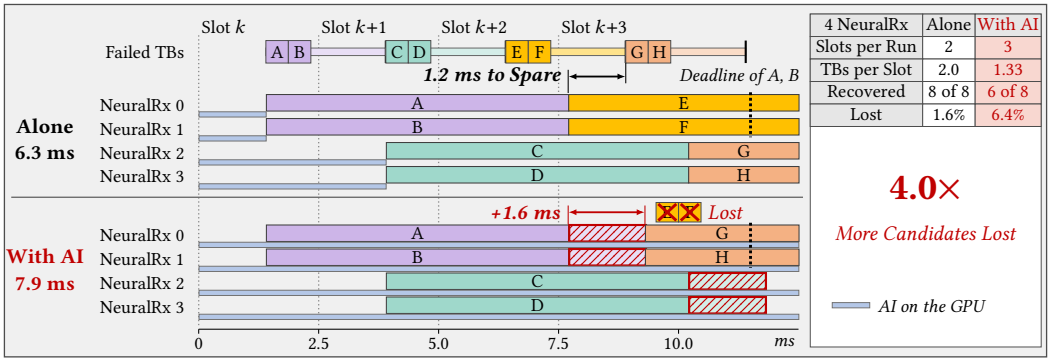



Fig. 3. Four neural receivers and two failed TBs in each of four uplink slots on a time axis. Every TB keeps its color, and the light bar behind it ends at its latest start time. Top: alone, a run ends 1.2 ms before the latest start time of E and F, and all eight TBs are recovered. Bottom: next to AI, a run is 1.6 ms longer (hatched) and still ends before its own recovery deadline, and E and F find every neural receiver busy. Right: TBs per slot that four neural receivers can take, and the lost candidates measured with 16 cells.

before the layer-1 deadline for all three, and both recoveries end 2.7-3.9 ms before their recovery deadline.

**Which TBs Are Recovered.** A failed TB becomes a *candidate* only if its slot has at most $K$ failed code blocks. The controller computes the *latest start time* of a candidate as its arrival plus the recovery deadline minus the time bound of the neural receiver, 11.5 ms minus 7.6 ms. It assigns candidates in order of latest start time to a free neural receiver. A candidate that passes its latest start time is dropped, and its TB is retransmitted. A neural receiver result counts only if the CRC passes, the payload is correct, and the run ends before the recovery deadline.

## 4 Recovery-Loss Model

This section explains how AI inference on the shared GPU costs recoveries and gives a model that computes the loss.

### 4.1 Where recoveries are lost

We run the recovery path next to AI inference under the MPS policies of Section 6 and classify every recovery that a policy loses against the run without AI on the same seed. Except for plain MPS without any protection, 83-99% of the lost recoveries are candidates that found no free neural receiver before their latest start time, and at most 17% are neural receiver runs that end after the recovery deadline. With low-priority AI and no cap, the neural receiver takes 7.9 ms at the median against 6.3 ms alone. More than 99% of its runs still end before the recovery deadline of 11.5 ms, and the policy loses 6.3% of the recoveries. Therefore, a rule that checks the deadline of the running neural receiver does not keep the recoveries.

### 4.2 Model

$N$ neural receivers serve the candidates of all cells, and an uplink slot arrives every $P = 2.5$ ms. A candidate of slot $k$ becomes known at time $kP + c$, where $c$ is the time that the conventional receiver takes (1.4 ms at the median). It must start on a neural receiver by $kP + L$, where $L = 3.9$ ms is the latest start time, and a candidate that finds no free neural receiver by then is lost. The neural receivers form a loss system [16]: a candidate cannot queue beyond $L$.

A run that starts at offset $s$ in slot $k$ and takes $S$ ends at $kP + s + S$. Its neural receiver can take a candidate of slot $k + m$ only if $s + S \le mP + L$. Thus, the run holds its neural receiver for

$$h = \left\lceil \frac{s + S - L}{P} \right\rceil \tag{1}$$

slots. Figure 3 shows the two cases for four neural receivers and two failed TBs per slot. A failed TB of slot $m$ can start between $mP + c$ and $mP + L$; with $c = 1.4$ ms these windows follow each other without a gap. With $s = 1.4$ ms, a run of $S = 6.3$ ms ends at 7.7 ms, 1.2 ms before the latest start time of the failed TBs of slot $k + 2$: it holds its neural receiver for two slots, and the TBs E and F of that slot take the neural receivers of A and B. A run longer than 7.5 ms ends after that latest start time and holds its neural receiver for three slots. Then, all four neural receivers are busy until E and F are dropped. $N$ neural receivers that hold $h$ slots each take at most $N/h$ candidates per slot: 2.0 with two slots and 1.33 with three for $N = 4$. With 16 cells, the run without AI loses 1.6% of the candidates, and low-priority AI without a cap loses 6.4%, 4.0× as many (Section 6.3). *AI inference costs recoveries when it moves runs from two slots to three*, not when it pushes a run past the recovery deadline. A neural receiver next to low-priority AI takes 7.3-8.0 ms: it meets the recovery deadline, and $h = 3$.

**Event Simulation.** The first form of the model replays the candidates of the run without AI (slot and time known) through $N$ servers. Its inputs are the run length of the neural receiver alone and next to AI, the rule that says when AI may run on a GPU, and the time AI takes to stop. It gives the lost candidates of a rule before the rule is run.

**Closed Form.** The second form is a Markov chain over the states of the neural receivers. A neural receiver is idle, frees in the current slot after a two-slot run or after a longer run, or is held for one, two, or three more slots. The chain state is the number of neural receivers in each of these states and the number of candidates $a$ of the current slot. The candidates take the idle neural receivers first and start at $s = c$. The remaining candidates take the neural receivers that free in the current slot and start at $s = \max(c, \text{free time})$, and the rest are lost. Equation 1 gives the slots that each run holds, with $c$ and $S$ drawn from their distributions. The number of candidates follows a transition matrix $T[a, a']$ measured in the run without AI: the failed TBs of a cell come in consecutive slots, and a cell with a candidate has one in the next slot with probability 0.19-0.26 against a mean of 0.145. We iterate the free-time distributions and the stationary distribution $\pi$ of the chain to a fixed point. The lost share is

$$\ell = \frac{\sum_x \pi(x)\ \max\big(0,\ a(x) - n_{\text{idle}}(x) - n_{\text{free}}(x)\big)}{\sum_x \pi(x)\, a(x)}, \tag{2}$$

where $n_{\text{idle}}(x)$ and $n_{\text{free}}(x)$ are the idle neural receivers and those that free in the slot in state $x$. The inputs are $N$, $T$, and the distributions of $c$ and $S$. The share $f$ of runs that hold their neural receiver for three slots is an output. A run that starts on an idle neural receiver takes three slots with probability $\Pr(c + S > 2P + L)$. A run that waits for a neural receiver to free starts late, and 93% of these runs take three slots; they are 8% of the runs at full load without AI.

### 4.3 What the model says about sharing

The model has two levels of loss. With 16 cells and four neural receivers, it gives 1.5% lost candidates when every run takes two slots and 6.2% when every run takes three; Section 6.3 compares both with measurements. The step between the levels is at the slack

$$2P + L - c - S = 5.0 + 3.9 - 1.4 - 6.3 = 1.2 \text{ ms}. \tag{3}$$

Table 3. Room of the neural receiver runs before they hold a third slot, and the share of runs that do.

| Setting | Policy | Median room (ms) | Three-slot runs | Candidates without a neural receiver |
|---|---|---|---|---|
| Full load, fixed MCS | No AI | 1.19 | 10.6% | 1.66% |
| | Antiphase | 0.67 | 11.2% | 1.68% |
| | *Priority* | 0.29 | 29.9% | 2.65% |
| | *Priority-70* | −1.18 | 86.7% | 6.38% |
| Rate control, 10% target | No AI | 0.77 | 21.2% | 6.52% |
| | Antiphase | 0.49 | 25.0% | 7.29% |
| | *Priority* | −0.05 | 52.2% | 9.87% |
| Eight two-user cells, 1% target | No AI | 0.91 | 27.3% | 5.23% |
| | Antiphase | 0.29 | 34.2% | 5.81% |
| | *Priority* | −0.20 | 65.5% | 8.18% |

Three consequences follow. First, running less AI next to the neural receiver is not enough: a share cap of 30% already adds 0.75-1.21 ms to the run. Second, beyond the slack the loss stops growing until a run takes four slots, 2.5 ms later, which is why a 70% share and no cap lose the same recoveries. Third, the criterion for a sharing rule is the extra run time of the neural receiver, not the GPU share of AI. This motivates Antiphase's rule: stop the AI of a GPU when its neural receiver starts, and keep the extra run time far below the slack.

**The Slack Differs by Run.** The 1.2 ms of Equation 3 is the slack of a run with the median $c$ and $S$. A run holds a third slot when $c + S > 2P + L$, and both terms vary: the conventional result of a slot can come late, and a run can wait for a free neural receiver. Table 3 gives the room $2P + L - (s + S)$ of every measured run, with $s$ its start offset. Without AI, 10.6% of the runs at full load already hold three slots, and a quarter have less room than the median stop latency of an AI piece (0.43 ms). The criterion for a sharing policy is therefore the share of three-slot runs against the server without AI, not a fixed increase of the run length. Antiphase keeps this share at 11.2% and *Priority* raises it to 29.9%. Antiphase uses part of the room in two ways: the conventional result arrives 0.14-0.35 ms later at the median, because AI units of the two smaller classes run next to decoding cells, and in 64-82% of the runs an AI unit is still on the GPU when the neural receiver starts (0.4 ms at the median, 1.4-1.5 ms at the 99th percentile), which lengthens the run by 0.3 ms per millisecond. With eight two-user cells at the 1% target this moves the three-slot share from 27.3% to 34.2%, the setting with the largest goodput loss of Antiphase. From the same records, a stop tail capped at 0.37 ms would give 32.9% and no tail at all 30.4%; a faster preemption mechanism can gain at most this much.

## 5 AI Admission in Stoppable Pieces

Figure 4 shows how Antiphase admits AI, with the values of a measured run at the instant the neural receiver of GPU 1 starts. ❶ *GPU State* holds, for every GPU, the TB on its neural receiver, the number of its cells that are decoding, and whether an AI piece is on the GPU. ❷ In every round and for every GPU, *AI Admission* writes a *grant*: the largest unit that may run, a budget of at most 4 ms of AI work, and a stop word. The AI workers of GPU 2 and GPU 3 hold a grant for units of up to 512 tokens and submit the units of their request one after another. ❸ When a neural receiver starts on a GPU, or a candidate waits for a neural receiver, the controller sets the stop word. On GPU 1 the unit on the GPU ends 0.45 ms later, and the four remaining units of the request wait until the neural receiver ends at 8.8 ms. GPU 0 runs a neural receiver and has no AI request.

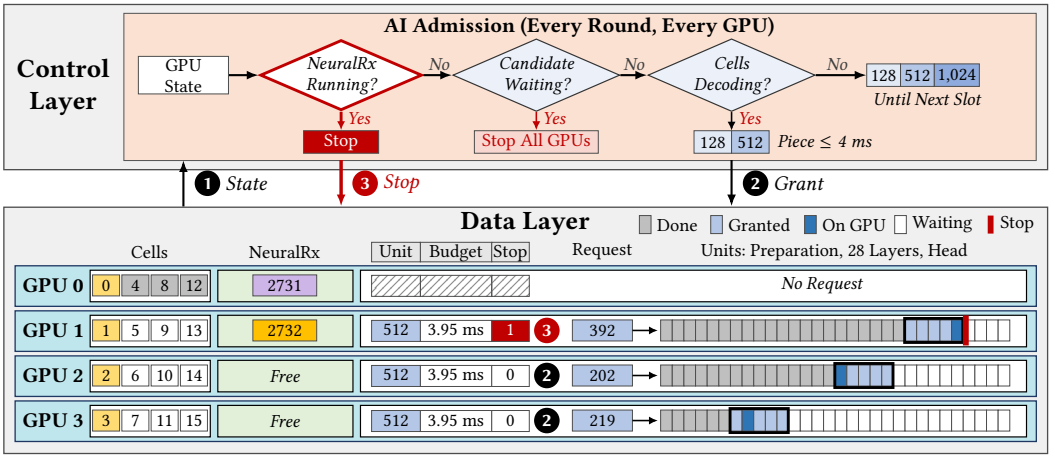



Fig. 4. AI admission with the values of a measured run at the instant the neural receiver of GPU 1 starts the TB of slot 2732 (Figure 2). Top: the decision of *AI Admission* for one GPU. Bottom: the cells (gray: decoding), the neural receiver, the grant, and the 30 units of the AI request of every GPU. Numbers mark the steps of the text.

Table 4. AI units and their time bounds (ms).

| Model | Unit | Bound |
|---|---|---|
| Qwen2.5-1.5B | one layer over 128 / 512 / 1,024 tokens | 0.37 / 0.84 / 1.33 |
| | one tenth of an output token (three layers) | 0.59-0.72 |
| Qwen2.5-3B | one layer over 128 / 512 / 1,024 tokens | 0.51 / 1.14 / 1.87 |
| BGE-base (text embedding) | one layer over a batch of 1 / 4 / 16 | 0.11 / 0.15 / 0.28 |
| ViT-base (image classification) | one layer over a batch of 1 / 4 / 16 | 0.13 / 0.19 / 0.40 |

**AI Units.** The AI worker splits every AI job into *units*. A unit is one GPU graph over fixed buffers with a time bound measured once, 1.1 times the 99.9th percentile of the unit alone. The prefill of an LLM request is one unit per layer over a chunk of 128, 512, or 1,024 tokens, an output token is ten units of three layers each, and an embedding or image request is one unit per layer over a batch of 1, 4, or 16 items. A request of up to 512 tokens is 30 units: preparation, 28 layers, and the output head. Table 4 lists the bounds. A unit belongs to one of three size classes by its bound: up to 0.45 ms, up to 1.2 ms, and above. The controller sees only the class of a unit and not the model or the kind of job.

**Low Priority.** The AI worker runs at a lower MPS priority than the radio processes [21] and has no cap on its share. Next to low-priority AI units of 1,024 tokens, the conventional receivers of four cells finish in 3.4 ms at the 99th percentile, against 2.9 ms alone, and the neural receiver takes 7.9 ms at the median, against 6.3 ms alone. Low priority alone keeps neither the recoveries (Section 4) nor, with the largest units, the layer-1 deadline at the 99.9th percentile.

**Grants.** The AI worker submits units only inside a piece that the controller grants. In every round and for each GPU, the controller picks the largest size class that meets all of the following and grants a piece of at most 4 ms of AI work:

- *Neural receiver.* No neural receiver runs on this GPU.
- *Waiting candidates.* No candidate is waiting for a free neural receiver.

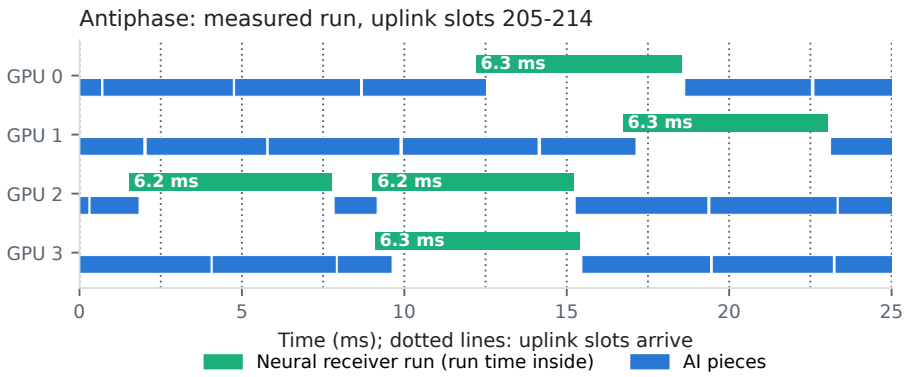



Fig. 5. Measured timeline of the four GPUs under Antiphase (16 cells, full load, 25 ms).

- *Conventional receivers.* The two smaller size classes may run next to the conventional receivers. The largest class runs only while no cell of this GPU is decoding, in a piece that ends before the next slot arrives.

If no class meets the conditions, the GPU gets no AI in this round. One controller round takes 15-57 $\mu$s.

**Stops.** A granted piece can stop. The worker submits the units of a piece in groups of at most 0.9 ms and reads a stop word of the controller between groups. The controller writes the stop word when a condition of the grant stops holding: a neural receiver starts on this GPU, or a candidate waits. The piece ends when the group on the GPU ends, 0.43 ms after the stop word at the median and 1.4 ms at the 99th percentile. Without stops, a piece runs to its end, and at low priority next to radio work a piece takes 2.3 ms at the median and 8.6 ms at the 99th percentile. A rule without stops must then keep AI off every GPU on which a neural receiver may start, or accept AI next to a running neural receiver (Section 6.8).

Figure 5 shows the rule on a measured run. A neural receiver starts on GPU 2 at 1.5 ms. The AI piece of that GPU ends 0.3 ms later, and AI starts again when the neural receiver ends at 7.8 ms. The neural receiver runs take 6.2-6.3 ms, as without AI. The AI workers of the GPUs without a running neural receiver continue.

**AI Requests.** Requests arrive in one list for the server. The controller predicts when a new request would finish on each GPU from the waiting work with the same or a shorter time limit and from the recent service rate of the GPU. It places the request on the GPU that finishes it first and rejects it if no GPU finishes it within 75% of its limit. Inside a piece, the worker runs first the unit of the job whose next result is due earliest after its remaining work. Embedding and image items wait for a batch for up to 30% of their limit. Every baseline uses the same placement, rejection, and order.

**Time Bounds.** The bounds come from measurements on the evaluation hardware with all cells running. The larger neural receiver takes 6.3-6.4 ms at the median next to four or five cells, and its bound is 7.6 ms. The bound sets the latest start time of a candidate: a larger bound has fewer runs that end after the recovery deadline and an earlier latest start time, and Section 6.11 chooses it with the model. A size class is allowed next to the conventional receivers if they finish before the layer-1 deadline at the 99.9th percentile next to continuously running low-priority units of that

class. With four cells per GPU, the receivers of the single-user cells finish in 3.65, 3.86, and 4.05 ms next to 128-, 512-, and 1,024-token units, and Antiphase allows the two smaller classes.

**Implementation.** The neural receiver path is the public TensorRT export of the NVlabs model followed by rate recovery, LDPC decoding, and CRC from cuPHY. We replace the least-squares channel estimation call with one multiplication of the pilot samples and call the cuPHY bit-level functions on buffers allocated once, which shortens the path of the real-time model from 3.08 to 2.36 ms with the same decoded TBs. The run-time path of the real-time model agrees with the NVlabs evaluation on 794 of 800 TBs. Antiphase is 4.1k lines of Python over cuPHY, TensorRT, and PyTorch.

## 6 Evaluation

### 6.1 Evaluation setup

**Hardware.** One node with four NVIDIA A100 80 GB GPUs under MPS and 64 CPU cores.

**Scope.** All measurements are end-to-end runs on the GPUs: cuPHY decodes every TB, the neural receiver decodes real received slots, and the AI worker runs real models. The slots are generated in advance; there is no radio unit and no layer-2 stack.

**Workloads.** Sixteen cells run on four GPUs, four per GPU, unless stated. One cell per GPU carries two users on the same 273 PRBs (16QAM, 20 code blocks per slot); the other cells carry one user with two layers. The two-user slots come from three channels of the NVlabs link simulation: low correlation at 2-3 dB, high correlation at 13-16 dB, and urban micro [1] at 4-6 dB. Every cell replays 512 slots (1.28 s). Both receivers run 20 LDPC iterations, the neural receiver is the larger NVlabs model, $K = 9$, and the recovery deadline is 11.5 ms. AI requests are Qwen2.5-1.5B [23] prefill requests of up to 1,024 tokens with lengths from BurstGPT [31] and a 200 ms limit, offered at 32 requests/s per GPU (53k tokens/s for the server) unless stated. A request that finishes after the limit does not count. Section 6.12 adds output tokens, a larger LLM, text embedding [33], and image classification [11]. Runs last 10-40 s; values are means over five seeds unless stated.

**Metrics.** *AI served* is the prompt tokens per second of requests that finish within the limit. *Recoveries kept* is the number of TBs that the neural receiver recovers before the recovery deadline as a share of the run without AI on the same seed; run-to-run variation moves it by a few tenths of a percent, and it can exceed 100%. *Recoveries lost* is 100% minus the recoveries kept. *Layer-1 misses* is the share of TBs whose conventional result passes the layer-1 deadline; in a run of 10-40 s most of them fall into the first seconds after the processes start (Section 6.7). *Lost candidates* is the share of candidates that find no free neural receiver by their latest start time.

**Baselines.** Every policy uses the same recovery path, the same AI requests, and the same placement and rejection of AI requests. The policies differ in when AI may use the GPU.

- *Static* gives the AI worker a fixed MPS share of 10%, a static split of the GPU between radio and AI. It is the largest share we measured that keeps the recoveries at full load; a 30% share loses 4.9% of them.
- *Priority* adds a low MPS priority for the AI worker, the mechanism that SMEC applies on an edge server [35], with a cap of 30% on the share. *Priority-70* and *Priority-max* use a cap of 70% and no cap; they serve more AI and lose recoveries.
- *Estimator* follows the design of YinYangRAN [25]. Every second it picks the largest AI share for which an estimator predicts that the reliability targets hold, given the histogram of the radio load of the last second. The estimator is a neural network trained offline with a quantile loss on runs at four steady loads and every share, with a seed that the evaluation does not use. The targets are 97% of the recoveries and 99.9% of the TBs within the layer-1 deadline. A

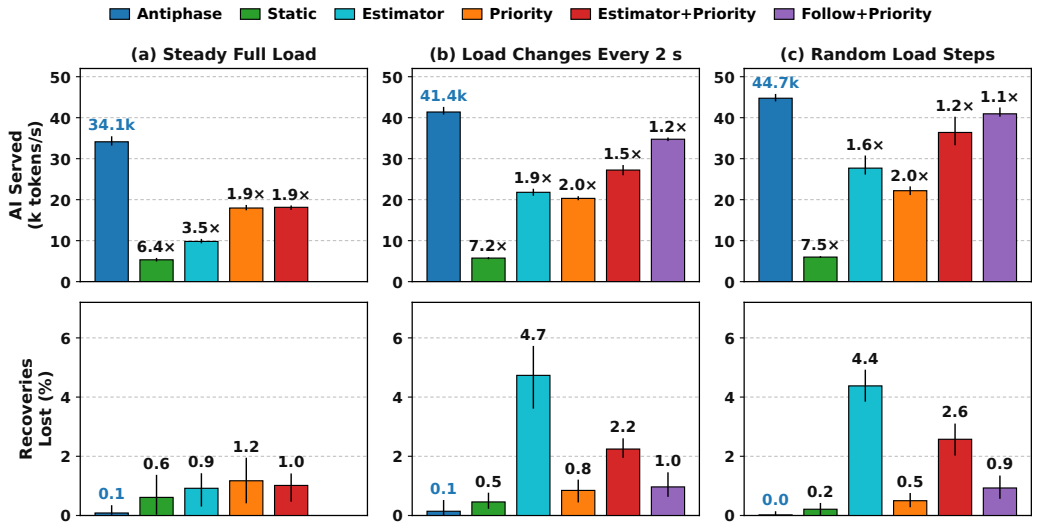



Fig. 6. AI served (top) and recoveries lost (bottom) under three load patterns (16 cells, four GPUs, five seeds; lines show the range over seeds). The number on a baseline is how many times more AI Antiphase serves than that baseline.

change of share stops AI for 0.27 s, the MPS reconfiguration time that YinYangRAN reports. *Estimator+Priority* runs it with low-priority AI.

- *Follow+Priority* is an idealized form of the same idea with low-priority AI: it keeps one AI worker per share loaded on every GPU and makes the worker of a lower share active when the radio load of the last 100 ms is higher. It knows the load at once and has no switching cost. *Follow* is the same without low priority.

In the tables, *Static* is the fixed 10% share and *Priority* is the 30% share with low-priority AI. The baselines are the mechanisms of these systems on our code, not the systems. The estimator baseline is favored in two ways: the radio work stays on the GPU during a reconfiguration, and no model is reloaded.

## 6.2 Summary of results

Figure 6 and Table 5 summarize the evaluation. At a steady full load (Figure 6a), Antiphase serves 34.1k tokens/s, which is 6.4× *Static* (5.3k), 3.5× *Estimator* (9.8k), and 1.9× *Priority* and *Estimator+Priority* (18.0k and 18.1k). Antiphase loses 0.1% of the recoveries, and the baselines lose 0.6-1.2%. When the load changes every 2 s (Figure 6b), Antiphase serves 41.4k tokens/s, 7.2× *Static*, 2.0× *Priority*, and 1.5× *Estimator+Priority*. The two estimator policies lose 4.7% and 2.2% of the recoveries, because their decision is one second behind the load, and Antiphase loses 0.1%. *Follow+Priority* knows the load without lag and serves 34.7k; Antiphase serves 1.2× more. With random load steps (Figure 6c), Antiphase serves 44.7k tokens/s, 7.5× *Static* and 1.1-2.0× the other baselines, and keeps 100.0% of the recoveries. In all three patterns Antiphase loses the fewest recoveries and serves the most AI of the policies in Figure 6. *Priority-70* and *Priority-max* serve 1.1-1.4× the AI of Antiphase and lose 2.7-6.3% of the recoveries (Section 6.4).

Table 5. Main results with 16 cells on four GPUs. Factors are Antiphase over the compared baseline.

| Result | Antiphase | Compared with | Factor |
|---|---|---|---|
| AI served, steady full load | 34.1k tokens/s | *Static* (5.3k) | 6.4× |
| | | *Priority* (18.0k) | 1.9× |
| | | *Estimator+Priority* (18.1k) | 1.9× |
| AI served, load changes every 2 s | 41.4k tokens/s | *Static* (5.7k) | 7.2× |
| | | *Estimator+Priority* (27.2k) | 1.5× |
| | | *Follow+Priority* (34.7k) | 1.2× |
| AI served, random load steps | 44.7k tokens/s | *Static* (6.0k) | 7.5× |
| | | *Estimator+Priority* (36.4k) | 1.2× |
| Recoveries lost | 0.0-0.1% | *Priority-70* (2.7-5.6%) | |
| Layer-1 latency added, 99.9th percentile | 0.04-0.06 ms | baselines (0.14-0.87 ms) | 2.8-21× less |
| AI served against a clairvoyant controller that was run | 87% | *Static* to *Priority* (14-46%) | 1.9-6.4× |
| Goodput lost by two-user cells, 42 settings | at most 0.5% | *Priority-max* (up to 8.2%) | |
| Loss model | 0.82 points error | 672 measured runs | |

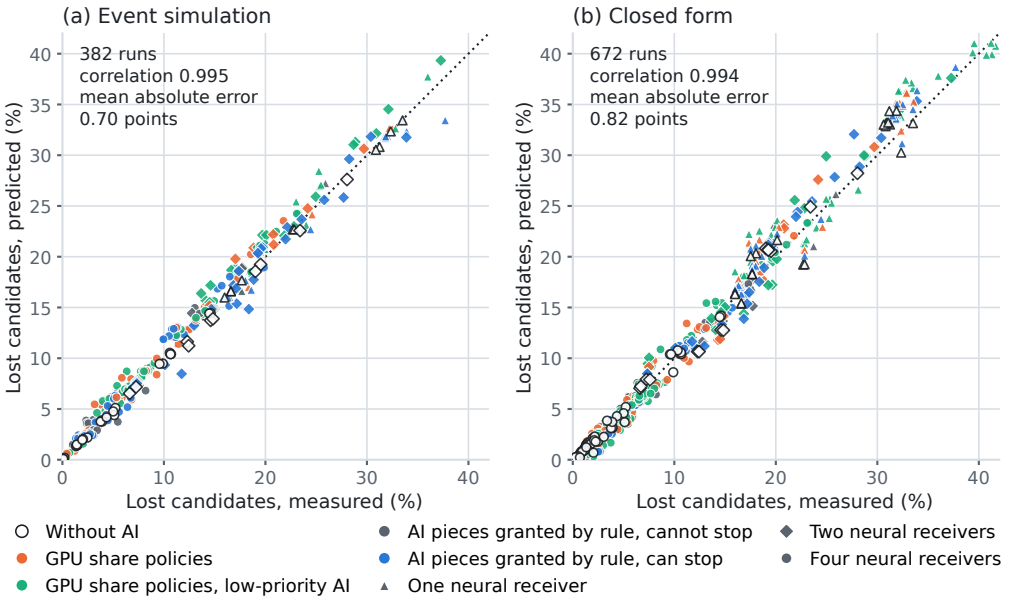



Fig. 7. Share of the candidates that find no free neural receiver, predicted against measured, for every run. GPU share policies: fixed share, share that follows the load, and share chosen by an estimator.

### 6.3 Model validation

**Takeaway.** The closed form predicts the lost candidates of 672 runs with a mean absolute error of 0.82 percentage points and has no fitted parameter.

We compare the closed form with 672 runs: one, two, and four neural receivers (106, 78, and 488 runs), 4-48 cells with one to twelve two-user cells, and 35 policies and settings. Figure 7 and Table 6 show the result. The closed form takes the distributions of $c$ and $S$ from the runs of the same policy with other seeds. Its mean absolute error is 0.48 points with four neural receivers, 1.45 with two, and 1.93 with one, over a range of 0-42% lost candidates. The share of three-slot runs

Table 6. Model against measurement: share of the candidates that find no free neural receiver. Errors are in percentage points. The first 412 runs existed when the model was built.

| Form | Inputs besides $N$ | Runs | Correlation | Mean absolute error | Mean error |
|---|---|---|---|---|---|
| Event simulation | arrivals without AI, $S$ alone and next to AI, rule | 334 | 0.995 | 0.77 | +0.46 |
| Closed form | $T$, distributions of $c$ and $S$ | 672 | 0.994 | 0.82 | +0.06 |
| Closed form, first runs | same | 412 | 0.993 | 0.95 | +0.07 |
| Closed form, runs measured after the model was final | same | 260 | 0.996 | 0.61 | +0.06 |
| Closed form with measured $f$ | $T$, $f$ | 412 | 0.993 | 0.92 | −0.14 |
| Closed form with measured $f$ and independent arrivals | mean arrival rate, $f$ | 412 | 0.972 | 2.87 | −2.76 |

Table 7. Lost candidates per setting: measured / closed form (%). In parentheses: share of three-slot runs, measured → closed form (%).

| Neural receivers, two-user cells | No AI | Antiphase | 70%, low-priority AI |
|---|---|---|---|
| 4, 2 | 0.0 / 0.1 (4 → 3) | 0.0 / 0.1 (7 → 5) | 1.3 / 1.0 (78 → 77) |
| 4, 4 | 1.6 / 1.6 (10 → 7) | 1.7 / 1.9 (11 → 10) | 6.2 / 5.7 (86 → 86) |
| 4, 6 | 4.7 / 4.4 (14 → 13) | 4.8 / 4.7 (17 → 16) | 11.4 / 12.0 (90 → 91) |
| 4, 8 | 10.1 / 10.5 (21 → 22) | 10.5 / 10.9 (23 → 24) | 19.7 / 20.4 (89 → 91) |
| 2, 2 | 7.0 / 7.5 (14 → 10) | 7.2 / 8.0 (16 → 15) | 14.1 / 14.2 (78 → 78) |
| 2, 4 | 19.3 / 20.7 (21 → 20) | 19.4 / 21.1 (24 → 24) | 28.8 / 30.0 (83 → 85) |
| 1, 1 | 16.8 / 17.3 (5 → 6) | 17.0 / 18.0 (13 → 13) | 24.1 / 22.7 (65 → 63) |
| 1, 2 | 30.9 / 33.0 (8 → 14) | 31.9 / 33.8 (19 → 20) | 40.3 / 40.0 (69 → 72) |

that it computes agrees with the measured share within 3.4 points on average. Giving the closed form the measured share does not make it more accurate. Assuming independent arrivals makes it underestimate the loss by more than half.

**Runs Measured After the Model Was Final.** The model has no fitted parameter. To test it on settings that we had not seen when we built it, we measured 260 more runs: other AI loads, AI requests in bursts, 20, 32, and 48 cells, a second latest start time, three more seeds with one GPU, and changing and bursty radio loads (Section 6.11). The mean absolute error on these runs is 0.61 points. It is 0.06-0.62 points per setting, except with one GPU, where the closed form is 2.5 points high, and with 2 s bursts on 32 cells, where it is 1.4 points low: the closed form describes the arrivals by one transition matrix and does not see the load change over seconds.

Table 7 gives the values per setting. In all eight settings, Antiphase loses at most 1.0 point more candidates than the run without AI, and *Priority-70* loses 1.3-9.6 points more. The model is least accurate for *Priority* on one and two neural receivers, where the closed form is 2.4-3.8 points high: a fixed share slows the neural receiver only while AI is active, and the closed form draws the run length of every run independently.

**Extra Run Time.** Figure 8 tests the slack of Equation 3. The closed form gives 1.5% lost candidates with no extra run time, 3.0% with 1.0 ms, and 5.9% with 1.4 ms, where 95% of the runs take three slots. When the conventional result is also 0.3 ms late, as it is next to AI, the step moves 0.3 ms to the left. The 13 measured policies lie between the two lines or within 0.3 points of them; one setting that allows AI next to a neural receiver while one other is free is 0.8 points below. Antiphase adds

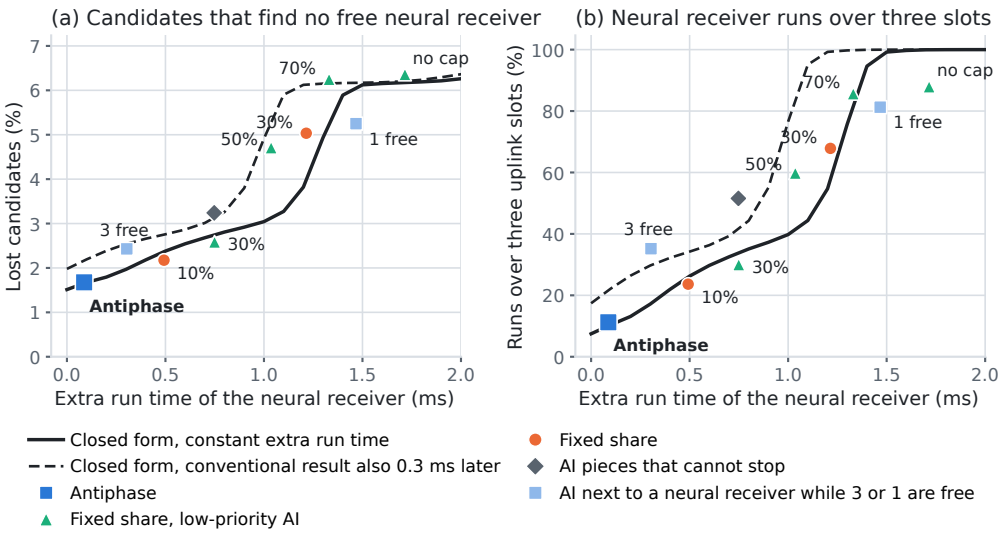



Fig. 8. Lost candidates and three-slot runs against the extra run time of the neural receiver (16 cells, four neural receivers). Lines: closed form with the arrivals and run lengths of the run without AI plus a constant extra run time. Points: measured policies at their median extra run time; labels give the MPS shares.

0.09 ms to the run of the neural receiver and loses 1.7% of the candidates, against 1.6% without AI. The 30% shares add 0.75-1.21 ms and lose 2.6-5.0%, and *Priority-70* adds 1.33 ms and loses 6.2%.

**A Bias of Slot Replay.** The model exposed a bias of our own method. Earlier runs replayed 128 slots per cell (0.32 s). The slots in which the failures of several cells coincide then repeat every 0.32 s, and the loss without AI was 1.45% and 3.76% for two seeds, which the closed form did not reproduce. With 512 slots the two seeds give 1.30% and 1.34%, and the recoveries kept by the baselines rise by 1.0-1.5 points. Every result in this section uses 512 slots.

**Predictions Before the Runs.** The comparisons above take $c$ and $S$ from runs of the same setting. Table 8 is a stricter test: six settings that had never been run, with nothing of the setting measured. Which slots of a two-user cell are candidates is fixed by the data set, the cell, and the seed; they are the same slots in every job, setting, and policy (all 2,317 candidates of a seed agree in two jobs and three policies). We therefore put the candidates of a new set of two-user cells together from earlier runs without AI, took $c$ and $S$ of each policy from its runs in the headline setting, and committed the 24 predicted losses to the repository before the runs started. Three GPUs had not been run before. The mean absolute error is 0.34 points, the largest is 1.21 points (the 30% share on two GPUs, where the closed form overestimates the three-slot runs, 39% against 30%), and the correlation is 0.998. The same procedure on five settings that had been measured before gives 0.73 points.

**Steps in the Recovery Deadline.** The model makes a prediction about the form of the loss. In $h = \lceil (s+S-L)/P \rceil$ the latest start $L$ is the recovery deadline minus the time limit, so a later deadline moves runs from three slots to two and from two to one, and the deadline at which this happens is later for a policy with longer runs. The loss should therefore fall in steps, at deadlines that differ by policy. We computed the candidates that are not recovered in time for thirteen deadlines from 11.0 to 14.0 ms, committed the values, and then ran the ten deadlines that had not been run (Figure 9; two seeds each; the default stays 11.5 ms). The closed form has no state for one-slot runs, so the computation is an event simulation with the candidates of the headline setting and, for each policy,

Table 8. Candidates that find no free neural receiver in six settings that had not been run: predicted before the run / measured. Three seeds for the first three settings and two for the others.

| GPUs, cells, two-user cells | No AI | Antiphase | 30%, low priority | 70%, low priority |
|---|---|---|---|---|
| 3, 12, 2 | 1.8 / 2.1% | 1.9 / 2.1% | 2.4 / 2.8% | 4.0 / 3.6% |
| 3, 12, 3 | 4.2 / 4.0% | 4.4 / 4.1% | 5.5 / 4.9% | 8.8 / 8.6% |
| 4, 16, 3 | 0.6 / 0.5% | 0.7 / 0.7% | 1.3 / 1.0% | 3.0 / 3.3% |
| 4, 16, 5 | 2.6 / 2.5% | 2.7 / 2.5% | 4.0 / 4.1% | 7.8 / 8.6% |
| 4, 16, 7 | 7.3 / 7.2% | 7.7 / 7.4% | 10.0 / 10.3% | 15.9 / 15.6% |
| 2, 8, 3 | 13.4 / 13.1% | 13.8 / 13.2% | 16.2 / 15.0% | 21.9 / 21.4% |

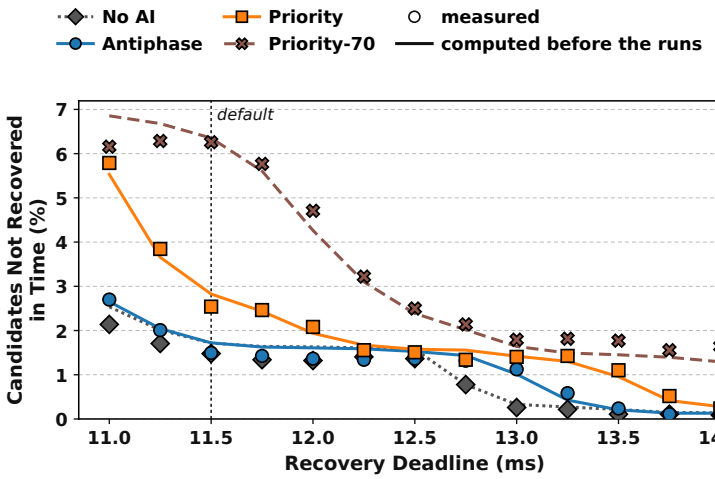



Fig. 9. Candidates that are not recovered in time against the recovery deadline (headline setting). Lines: computed and committed before the runs. Markers: measured. The loss falls in steps, at deadlines that differ by policy.

the distributions of $c$ and $S$ measured at 11.5 ms. The 40 new values have a mean absolute error of 0.16 points, a mean error of +0.02 points, and a largest error of 0.70 points.

The steps are where the model puts them. Without AI the loss is 2.1% at 11.0 ms, flat at 1.3-1.5% from 11.5 to 12.5 ms, where the runs hold two slots, and 0.3% at 13.0 ms, where 60% of the runs are predicted to hold one. The three-slot runs fall below half at 11.25 ms for *Priority* and at 12.0 ms for *Priority-70*; the loss falls below 0.5% at 13.0 ms without AI, at 13.5 ms for Antiphase, and at 14.0 ms for *Priority*, and *Priority-70* stays at 1.6%. The default deadline lies on the flat part of Antiphase: from 11.5 to 12.5 ms its loss is within 0.1 points of the server without AI, while the shares are still leaving their three-slot range. At the edges of a step Antiphase loses more than the server without AI, 0.3-0.8 points at 11.0-11.25 ms and at 12.75-13.25 ms: the AI left after a stop lengthens a run by 0.1-0.2 ms, which moves the step by about 0.25 ms. The computation had this difference at the second edge (0.1-0.7 points). The AI served does not move with the deadline: over the thirteen deadlines Antiphase serves 33.4k-34.5k tokens/s and keeps 99.2-100.0% of the recoveries, *Priority* 17.3k-18.3k and 94.9-99.7%, and *Priority-70* 41.1k-42.6k and 93.9-98.4%.

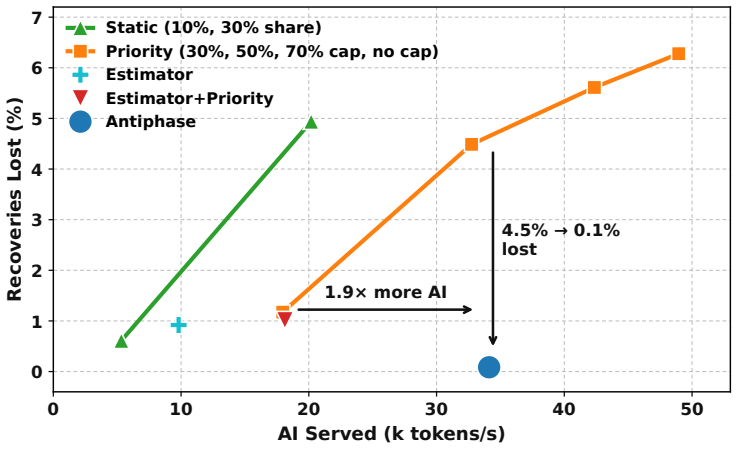



Fig. 10. AI served against recoveries lost at a steady full load (16 cells, four GPUs). A baseline that takes a larger share moves up along its line.

Table 9. Steady full load, 16 cells, 53k tokens/s offered (range over seeds in parentheses; five seeds, rows marked * two).

| Policy | AI tokens/s within 200 ms | Recoveries kept | Layer-1 misses |
|---|---|---|---|
| Antiphase | 34.1k (33.4-35.3k) | 99.9% (99.7-100.3%) | 0.036% |
| Fixed 10% | 5.3k (5.1-5.6k) | 99.4% (98.7-99.9%) | 0.032% |
| Fixed 30%* | 20.2k (20.1-20.3k) | 95.1% (94.9-95.2%) | 0.051% |
| Fixed 30%, low-priority AI | 18.0k (17.6-18.5k) | 98.8% (98.1-99.6%) | 0.037% |
| Fixed 50%, low-priority AI* | 32.7k (32.5-33.0k) | 95.5% (95.5-95.5%) | 0.040% |
| Fixed 70%, low-priority AI | 42.4k (41.1-43.4k) | 94.4% (93.6-95.1%) | 0.037% |
| Low-priority AI, no cap | 49.0k (47.5-49.9k) | 93.7% (93.4-94.5%) | 0.044% |
| Share chosen by an estimator | 9.8k (9.5-10.3k) | 99.1% (98.6-99.7%) | 0.019% |
| Share chosen by an estimator, low-priority AI | 18.1k (17.7-18.4k) | 99.0% (98.6-99.5%) | 0.018% |
| No AI | 0 | 100% | 0.032% |

### 6.4 AI served at a steady full load

**Takeaway.** At a steady full load Antiphase serves 6.4× the AI of *Static* and 1.9× that of the best existing mechanism that keeps the recoveries, and it keeps 99.9% of the recoveries.

Figure 10 and Table 9 show AI served against recoveries with all 16 cells at full load. The run without AI recovers 187 TBs per second. A baseline cannot reach Antiphase by taking more GPU time. Raising the cap of *Priority* from 30% to 50% brings its AI to 32.7k tokens/s, close to Antiphase, and its lost recoveries from 1.2% to 4.5%; Antiphase serves 34.1k and loses 0.1%. A 30% share for *Static* serves 20.2k and loses 4.9%.

**Recoveries and Layer 1.** Antiphase keeps 99.9% of the recoveries. It misses the layer-1 deadline for 0.036% of the TBs against 0.032% without AI, and the conventional receivers finish in 3.14 ms at the 99.9th percentile against 3.10 ms without AI and 3.51 ms under *Priority-70*.

**AI Served.** Antiphase serves 34.1k tokens/s. Among the existing mechanisms that keep at least 98.8% of the recoveries, *Estimator+Priority* and *Priority* serve the most, 18.1k and 18.0k; Antiphase serves 1.9× more. It serves 6.4× more than the fixed 10% share and 3.5× more than the estimator

Table 10. Changing load, 16 cells, five seeds. Each cell shows AI tokens/s within 200 ms and recoveries kept.

| Policy | Full and half load, every 2 s | Random steps, 25-100% |
|---|---|---|
| Antiphase | 41.4k, 99.9% | 44.7k, 100.0% |
| Fixed 10% | 5.7k, 99.5% | 6.0k, 99.8% |
| Fixed 30%, low-priority AI | 20.3k, 99.2% | 22.2k, 99.5% |
| Fixed 70%, low-priority AI | 46.9k, 96.0% | 48.6k, 97.3% |
| Share follows the load | 23.8k, 99.2% | 30.0k, 98.6% |
| Share follows the load, low-priority AI | 34.7k, 99.0% | 40.9k, 99.1% |
| Share chosen by an estimator | 21.8k, 95.3% | 27.7k, 95.6% |
| Share chosen by an estimator, low-priority AI | 27.2k, 97.8% | 36.4k, 97.4% |

without low priority. At a steady full load the estimator settles on a share of 20%, or 30% with low-priority AI, and equals that fixed share. The 70% share with low-priority AI and *Priority-max* serve 24% and 44% more AI than Antiphase and lose 5.6% and 6.3% of the recoveries.

### 6.5 AI served under a changing load

**Takeaway.** When the radio load changes, Antiphase serves 7.2-7.5× the AI of a fixed 10% share and 1.2-1.5× that of *Estimator+Priority*, and it keeps 2.1-2.6 points more recoveries than that policy.
**Alternating Load.** The radio load alternates every two seconds between full load and each cell active with probability 0.5. Figure 6 and Table 10 show the result. Antiphase serves 41.4k tokens/s and keeps 99.9% of the recoveries, with layer-1 misses of 0.019% against 0.018% without AI. This is 7.2× the AI of *Static*. The estimator serves 21.8k tokens/s and keeps 95.3%: its decision rests on the load of the last second, and its shares run one second behind the load, at 70% for half of every full-load period. With low-priority AI it serves 27.2k at 97.8%; Antiphase serves 1.5× more and keeps 2.1 points more recoveries. The share that follows the load at once and without a switching cost serves 23.8k at 99.2%, and 34.7k at 99.0% with low-priority AI; Antiphase serves 1.2× more. *Priority-70* serves 13% more AI than Antiphase and loses 4.0% of the recoveries.
**Alternation Interval.** With a change every 0.2 s and every 5 s (two seeds), Antiphase serves 42.5k and 41.2k tokens/s and keeps 99.8% of the recoveries in both. The policies that change the share with the load lose more recoveries as the interval gets shorter. The estimator decides every second: it keeps 94.0% with a change every 0.2 s, where it misses the layer-1 deadline for 0.24% of the TBs, and 97.9% with a change every 5 s; with low-priority AI it keeps 96.6% and 98.7%. The share that follows the load judges the load over 100 ms and keeps 97.3% with a change every 0.2 s, 97.6% with low-priority AI.
**Random Load Steps.** The share of active cells jumps between 25, 50, 75, and 100% at random times 0.5-1.5 s apart for 40 s. Antiphase serves 44.7k tokens/s and keeps 100.0% of the recoveries. The estimator with low-priority AI serves 36.4k at 97.4%, and the share that follows the load with low-priority AI serves 40.9k at 99.1%. The difference comes from the periods of full load. There Antiphase serves 34.9k tokens/s and keeps 100.0%. The share that follows the load drops to its lowest share and serves 3.9k, or 20.4k with low-priority AI. *Priority-70* and *Estimator+Priority* serve 44.0k and 35.4k and keep 94.3% and 94.7%.
**Bursts.** When every cell switches on and off by itself with a mean busy run of 20 ms or 2 s (half load on average, two seeds), the neural receivers are needed half as often as at full load. With 16 cells, Antiphase serves 49.6k and 48.0k tokens/s and keeps 100.1% and 99.8% of the recoveries, and the 70% share with low-priority AI serves 51.1k and 50.4k and keeps 99.5% and 98.6%: at this load low priority alone loses few recoveries. With 32 cells, Antiphase serves 33.5k and 34.4k tokens/s at 99.7% and 99.1%, 1.9× *Priority*, and *Priority-70* serves 22-23% more and loses 3.8-4.6% of the

Table 11. Uplink goodput with the retransmission schedule applied to the TB records of the runs of Table 9 (two-user cells).

| Policy | Goodput against no recovery path | Retransmissions per 100 TBs | Delay (ms), mean / 99th percentile |
|---|---|---|---|
| No recovery path | 0 | 16.9 | 5.77 / 12.0 |
| Recovery path, no AI | +5.61% | 10.7 | 5.69 / 19.5 |
| Antiphase | +5.63% | 10.7 | 5.71 / 19.5 |
| Fixed 10% | +5.60% | 10.7 | 5.72 / 19.5 |
| Share chosen by an estimator, low-priority AI | +5.58% | 10.7 | 5.74 / 19.5 |
| Fixed 30%, low-priority AI | +5.57% | 10.7 | 5.74 / 19.5 |
| Fixed 70%, low-priority AI | +5.31% | 11.0 | 5.78 / 19.5 |
| Low-priority AI, no cap | +5.27% | 11.1 | 5.80 / 19.5 |

recoveries. With 2 s runs, several cells of a GPU are busy for seconds, and the layer-1 misses are 0.25% with 16 cells and 1.1% with 32 cells without AI. Every policy with AI misses more (0.26-0.31% and 1.3-1.9%), and we do not compare the policies on the layer-1 deadline in this setting.

### 6.6 What a recovery is worth

**Takeaway.** The recovery path raises the goodput of two-user cells by 5.6% at a fixed MCS and by 11-40% under closed-loop rate control on a high-correlation channel. Antiphase loses at most 0.5% of the goodput in 42 settings, where the other policies lose up to 8.2%.

**Goodput.** We apply the retransmission schedule to the per-TB records of the runs. A TB that the neural receiver recovers needs no retransmission; a TB that is not a candidate or gets no neural receiver is retransmitted 7.5 ms after its slot, as without a recovery path; a TB whose neural receiver run fails is retransmitted 7.5 ms later, in the uplink slot at 15.0 ms (Section 3.2). A retransmission succeeds with probability 0.95 and takes one uplink slot from new data. Table 11 shows that the recovery path raises the goodput of two-user cells by 5.6% and cuts the retransmissions per 100 TBs from 16.9 to 10.7. The policies differ little: one percent of lost recoveries is 0.06 points of goodput, and Antiphase and *Priority-max* are 0.36 points apart. Over eight settings (full, alternating, and random load, eight two-user cells, two GPUs, and 20, 32, and 48 cells), the recovery path adds 4.6-5.9% of goodput, Antiphase keeps that gain within 0.03 points, and one percent of lost recoveries costs 0.05-0.06 points. The largest gap is with eight two-user cells, where Antiphase and *Priority-70* are 0.59 points apart (+5.26% against +4.67%). Thus, the result of Antiphase in goodput is that it keeps the gain of the recovery path while serving 34.1k tokens/s, where the existing mechanisms that keep the same gain serve 5.3k-18.1k. The 99th-percentile delay grows from 12 to 19.5 ms with the recovery path, because the retransmission of a failed recovery is 7.5 ms late; the mean delay falls from 5.77 to 5.69 ms. The goodput does not depend on the slot that carries the retransmission. A user of a two-user cell has 2.5 HARQ processes in use on average and at most six with and without the recovery path, within the 16 that NR allows [4].

**Link Adaptation.** A recovery path may allow a higher modulation and coding scheme (MCS). We decode the same slots with both receivers for MCS 10-16 at fixed Es/No points on the three channels of the workload (Table 12; Figure 11 shows the low-correlation channel). The larger model outputs 16QAM soft bits, and one engine serves every code rate. On the low-correlation channel, the recovery path moves the point of 10% first-transmission errors by 0.4-0.5 dB, where one MCS step is 0.8-1.1 dB. A rate controller with a 10% target picks one MCS step higher at three of seven points, and the goodput rises by 5.2%, against 2.9% with the MCS unchanged. On the high-correlation channel, the error rate of the conventional receiver falls slowly with the SNR, and the recovery

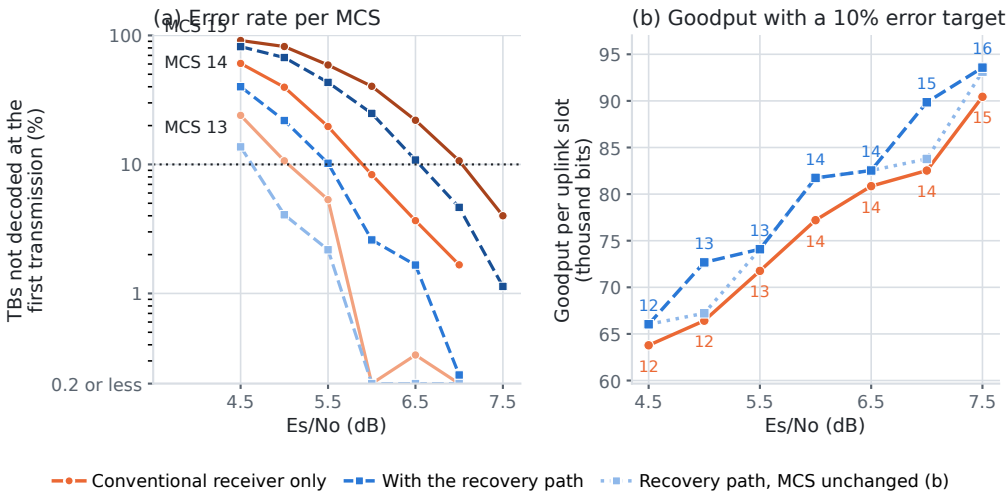



Fig. 11. Link adaptation with the recovery path (two-user cell, low-correlation channel, 300 TBs per point). Dotted line in (a): 10% target. Numbers in (b): MCS chosen.

Table 12. Link adaptation with the recovery path on the three channels of the workload (two-user cell, MCS 10-16, 200-600 TBs per point). Gains are in goodput with a 10% target for first-transmission errors.

| Channel | Es/No (dB) | Shift of the 10% point (dB) | Gain, MCS unchanged | Gain, MCS chosen with the recovery path | Points with a higher MCS |
|---|---|---|---|---|---|
| Low correlation | 4.5-7.5 | 0.4-0.5 | +2.9% | +5.2% | 3 of 7 |
| High correlation | 12-24 | 2.5-4.0 | +5.2% | +11.8% | 6 of 7 |
| Urban micro | 4-14 | 0.2-1.5 | +1.6% | within the statistical error | none confirmed |

path moves the 10% point by 2.5-4.0 dB. The rate controller picks one MCS step higher at six of seven points, and the goodput rises by 11.8%, against 5.2% with the MCS unchanged; at four of these points the conventional receiver fails 16-19% of the TBs at the higher MCS. On the urban micro channel, the neural receiver decodes few of the failed TBs: the goodput rises by 1.6-2.1%, and no MCS step is outside the statistical error. With the target itself as a parameter, the conventional receiver alone is best at a 10% target on the low-correlation channel, and with the recovery path the best target is 20-30%, 5.5% above; on the high-correlation channel the best targets are 7.5% apart. A higher MCS raises the neural receiver demand, which takes GPU time from AI (Table 25).

**Closed-Loop Rate Control.** The results above fix the MCS. We add an outer loop [8] to the two-user cells. A cell holds the slots of MCS 10-16 on the high-correlation channel at 16 dB and picks the MCS of every slot from a pointer over the levels. The pointer falls by 0.25 levels for every TB that needs a retransmission and rises by $0.25\,\tau/(1-\tau)$ for every other TB, where $\tau$ is the target share of TBs that need a retransmission; a TB that the neural receiver recovers before the recovery deadline needs none. The outcome of a slot reaches the loop 15 ms later. Figure 12 and Table 13 report the goodput of the two-user cells, the new data per uplink slot of a user, over 18 s per run.

**The Recovery Path Is Worth More in Closed Loop.** With a 10% target on four GPUs, the loop without a recovery path settles at a mean MCS of 12.5 and the loop with it at 13.5: the goodput rises by 12.0%. With 3% and 1% targets the gain is 24-31%. On this channel the error rate of the

Table 13. Closed-loop link adaptation: two-user cells on the high-correlation channel at 16 dB choose the MCS of every slot by an outer loop. Target: share of TBs that need a retransmission. Policy cells: AI served (tokens/s) / goodput of the two-user cells against the same server without AI. Rows without a cell count have one two-user cell per GPU.

| | Recovery path, no AI | | | | Low-priority AI | | |
|---|---|---|---|---|---|---|---|
| Server, target | Goodput gain | Mean MCS, without / with | Antiphase | Fixed 10% | 30% share | 70% share | No cap |
| 4 GPUs, 10% | +12.0% | 12.5 / 13.5 | 11.4k / 0.0% | 4.6k / −0.1% | 11.5k / −0.3% | 25.4k / −0.7% | 30.1k / −1.0% |
| 4 GPUs, 3% | +31.1% | 10.7 / 12.8 | 19.8k / +0.2% | 5.0k / +0.2% | 14.0k / −0.1% | 32.8k / −0.7% | 39.4k / −1.4% |
| 4 GPUs, 1% | +31.0% | 10.0 / 12.1 | 33.7k / −0.1% | 5.6k / −0.2% | 18.0k / −0.4% | 41.9k / −0.8% | 49.0k / −2.3% |
| 8 two-user cells, 3% | +25.9% | 10.7 / 12.4 | 6.8k / −0.2% | 4.4k / −0.7% | 10.3k / −1.2% | 21.5k / −2.0% | 28.2k / −3.9% |
| 8 two-user cells, 1% | +26.5% | 10.0 / 11.8 | 17.0k / −0.3% | 5.0k / −1.1% | 14.3k / −2.3% | 34.1k / −3.7% | 45.5k / −8.2% |
| 2 GPUs, 10% | +10.9% | 12.5 / 13.4 | 7.0k / −0.1% | 2.2k / −0.3% | 5.9k / −0.3% | 14.7k / −0.8% | 17.0k / −1.0% |
| 2 GPUs, 3% | +26.0% | 10.8 / 12.6 | 11.9k / +0.1% | 2.4k / −0.2% | 7.3k / −0.4% | 18.2k / −1.1% | 21.1k / −1.6% |
| 2 GPUs, 1% | +29.2% | 10.0 / 11.9 | 17.6k / 0.0% | 2.6k / −0.1% | 9.0k / −0.9% | 21.9k / −3.6% | 24.7k / −4.6% |
| 1 GPU, 1% | +24.4% | 10.0 / 11.7 | 9.0k / 0.0% | 1.1k / 0.0% | 4.5k / −0.3% | 10.2k / −2.4% | 11.5k / −2.5% |

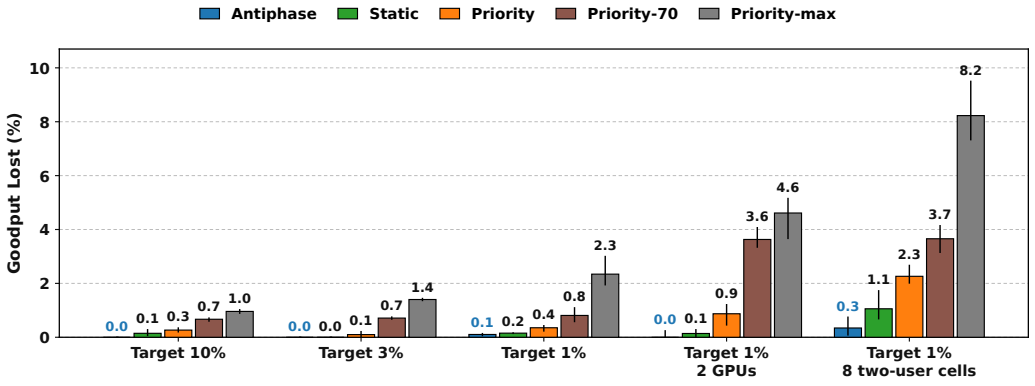



Fig. 12. Goodput of the two-user cells that each policy loses against the same server without AI, with an outer-loop rate controller (target: share of TBs that need a retransmission; lines show the range over seeds).

conventional receiver falls slowly with the MCS (1.4% at MCS 10), and the loop without a recovery path gives up two MCS levels to meet the target.

**Lost Recoveries Become a Lower MCS.** The share of TBs that need a retransmission equals the target under every policy. A policy that loses recoveries spends more slots at the lower of two neighbouring levels: the mean MCS falls by at most 0.64 levels, not by one level. One percent of lost candidates costs 0.09% of goodput at the 10% target, 0.15-0.37% at 3%, and 0.5-0.8% at 1%, against 0.06 points at a fixed MCS. The error rate after recovery changes by 10.3 points per MCS level near the 10% target and by 2.0 points near 1%: the same rise of the error rate moves the loop five times as far.

**Where the Loss Is Visible.** With a 10% target, the policies are within 1.0% of each other in goodput on two and four GPUs. With a 1% target and a neural receiver demand near the capacity, the loss grows: with eight two-user cells, *Priority* and *Priority-70* lose 2.3% and 3.7%, and *Priority-max* loses 8.2%; with two GPUs they lose 0.9%, 3.6%, and 4.6%. Antiphase stays within 0.4% of the server

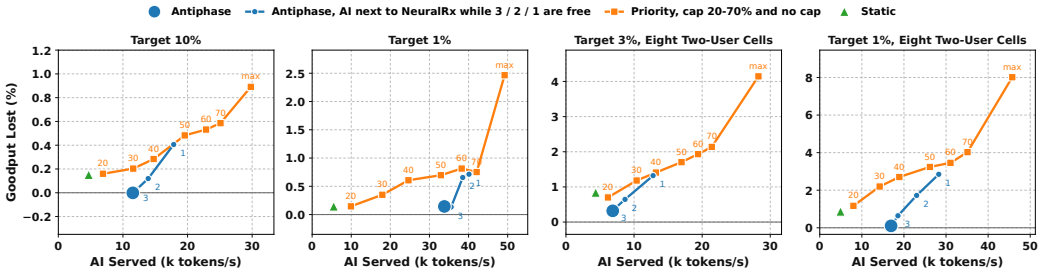



Fig. 13. AI served against the goodput of the two-user cells that is lost, with the outer loop (three seeds; all policies of a panel run in one job). Orange: low-priority AI with a cap of 20-70% and without a cap. Blue: Antiphase, and its settings that allow AI next to a neural receiver while 3, 2, or 1 neural receivers are free.

without AI in all nine settings. In these two settings the 30% share with low-priority AI serves less AI than Antiphase (14.3k against 17.0k, 9.0k against 17.6k) and loses more goodput.

**What a Share Pays.** With a 10% target, the loop raises the MCS until the neural receivers run for 68% of the time: 28.7% of the slots of a two-user cell have a candidate, and 6.7% of the candidates find no neural receiver without any AI. Antiphase serves AI in the remaining time, 11.4k tokens/s, and keeps the goodput. A share runs AI next to the running neural receivers. The 30% share with low-priority AI serves 11.5k and pays 0.3% of the goodput, the 70% share serves 25.4k and pays 0.7%, and low priority alone serves 30.1k and pays 1.0%. The same three shares pay 2.3%, 3.7%, and 8.2% with eight two-user cells at the 1% target: the price of a share is known only after the load is. The AI that Antiphase serves follows the time in which the neural receivers are idle: 64% at the 1% target (33.7k, 1.9× the 30% share), 46% at the 3% target (19.8k, 1.4×), and 32% at the 10% target, where it equals the AI of a 30% share that runs all the time. With eight two-user cells and a 3% target, Antiphase serves 6.8k and stays within 0.2%, and the 30% share serves 10.3k and pays 1.2%. A server without the recovery path serves all 53.0k offered tokens/s with low-priority AI and has 10.6% less goodput in the two-user cells. With a step of 0.5 levels in the outer loop, the losses of the 70% share and of *Priority-max* are 0.4% and 0.6% at the 10% target, and those of the three low-priority policies are 1.8%, 3.1%, and 6.5% with eight two-user cells at the 1% target; the order of the policies is the same.

**No Cap Replaces the Rule.** A share with a well-chosen cap might keep the radio. We sweep the cap of the low-priority share from 20% to no cap in four settings (Figure 13). A higher cap serves more AI and loses more goodput in every setting: from the 20% cap to no cap, the loss grows from 0.2% to 0.9% at the 10% target, from 0.2% to 2.5% at the 1% target, and from 1.2% to 8.0% with eight two-user cells at the 1% target. The cap that keeps the goodput within 0.3% depends on the setting: it is 30% at the 10% target and 20% at the 1% target with four two-user cells, and with eight two-user cells the fixed 10% share already loses 0.8-0.9%. Antiphase loses 0.0-0.3% in the four settings with one configuration. In no setting does a cap serve at least the AI of a setting of Antiphase at no more loss; §6.11 repeats the 70% cap at later recovery deadlines. Reading the AI at a loss of at most 0.25% from the lines between the measured points, Antiphase serves 1.2×, 2.4×, 2.2×, and 7.8× the AI of the caps. The settings of Antiphase that allow AI next to a neural receiver trade goodput for AI in the same way as a cap and from a lower loss; the default allows none.

**Other SNRs, Another Channel, and a Changing Channel.** The results above use one channel at one Es/No. We generate the slots of MCS 10-16 for the high-correlation channel at 14 and 20 dB and for the low-correlation channel at 6 dB, and we let the Es/No of a two-user cell change: a cell stays 2 s at 14, 16, or 20 dB and then moves to another value at random, at times that differ

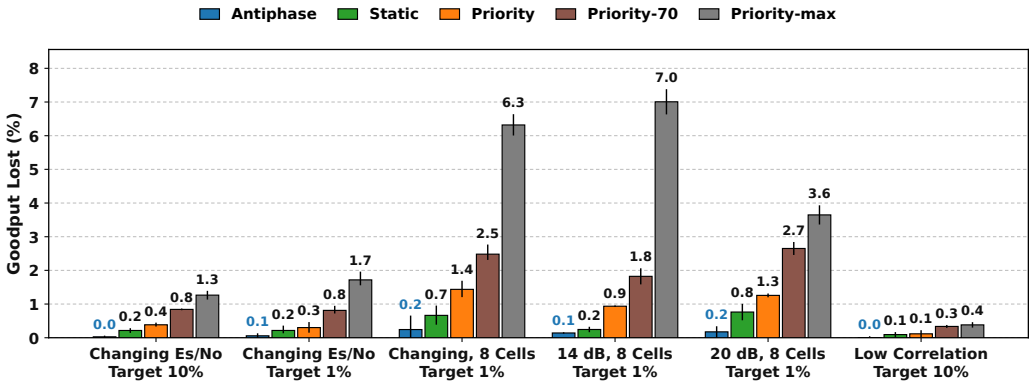



Fig. 14. Goodput of the two-user cells that each policy loses in other settings of the outer loop. Changing Es/No: every two-user cell moves between 14, 16, and 20 dB and stays 2 s at a value. 8 Cells: eight two-user cells. Low correlation: the low-correlation channel at 6 dB. Lines show the range over seeds.

Table 14. Closed-loop link adaptation in twelve more settings. Cells: AI served (tokens/s) / goodput of the two-user cells against the same server without AI. Busy: share of the time in which the neural receivers run without AI. Rows without a cell count have four two-user cells.

| Setting, target | Gain | Busy | Antiphase | Fixed 10% | 30% share | 70% share | No cap |
|---|---|---|---|---|---|---|---|
| Changing Es/No, 10% | +12.5% | 70% | 10.4k / 0.0% | 4.7k / −0.2% | 11.3k / −0.4% | 24.6k / −0.8% | 30.1k / −1.3% |
| Changing Es/No, 1% | +28.4% | 35% | 34.4k / −0.1% | 5.6k / −0.2% | 18.1k / −0.3% | 42.9k / −0.8% | 49.1k / −1.7% |
| Changing Es/No, 8 cells, 1% | +25.5% | 58% | 17.4k / −0.2% | 5.1k / −0.7% | 13.9k / −1.4% | 33.0k / −2.5% | 43.8k / −6.3% |
| 14 dB, 10% | +13.9% | 80% | 6.1k / −0.1% | 4.3k / −0.4% | 10.0k / −0.7% | 20.7k / −1.3% | 25.8k / −2.5% |
| 14 dB, 1% | +18.2% | 39% | 31.5k / −0.1% | 5.4k / −0.1% | 17.0k / −0.2% | 41.2k / −0.6% | 49.2k / −2.2% |
| 14 dB, 8 cells, 1% | +15.6% | 60% | 15.5k / −0.1% | 4.9k / −0.2% | 13.3k / −0.9% | 31.3k / −1.8% | 45.2k / −7.0% |
| 20 dB, 10% | +10.8% | 55% | 18.7k / 0.0% | 5.0k / −0.1% | 13.5k / −0.1% | 31.2k / −0.4% | 36.2k / −0.5% |
| 20 dB, 1% | +40.2% | 29% | 38.5k / +0.1% | 5.7k / +0.1% | 19.3k / 0.0% | 45.3k / −0.5% | 50.5k / −0.6% |
| 20 dB, 8 cells, 1% | +39.5% | 58% | 17.2k / −0.2% | 5.0k / −0.8% | 13.5k / −1.3% | 31.9k / −2.7% | 38.3k / −3.7% |
| Low correlation, 10% | +4.3% | 52% | 21.7k / 0.0% | 5.1k / −0.1% | 14.2k / −0.1% | 33.3k / −0.3% | 39.1k / −0.4% |
| Low correlation, 1% | +8.4% | 16% | 47.8k / 0.0% | 6.0k / 0.0% | 22.9k / 0.0% | 50.4k / 0.0% | 52.6k / 0.0% |
| Low correlation, 8 cells, 1% | +8.6% | 33% | 36.2k / 0.0% | 5.5k / 0.0% | 18.5k / 0.0% | 43.7k / −0.2% | 49.6k / −0.2% |

between the cells. The outer loop does not know the channel and follows the outcomes. Table 14 and Figure 14 show the twelve settings. Antiphase loses at most 0.2% of the goodput in all of them, also when the Es/No changes; with the nine settings of Table 13 this makes 21 settings with a loss of at most 0.4%. The shares lose by setting: the 30% share 0.0-1.4%, the 70% share 0.0-2.7%, and low priority alone 0.0-7.0%. The large losses are those of the high-correlation channel with eight two-user cells, at 14 dB, at 20 dB, and with a changing Es/No. On the low-correlation channel the recovery path adds 4-9% and a lost recovery costs little: no share loses more than 0.4%, and Antiphase serves 1.5-2.1× the AI of the 30% share because the neural receivers run for 16-52% of the time. The AI that Antiphase serves follows the idle time of the neural receivers in all settings: against the 30% share it serves 0.6× at 20% idle time (14 dB at the 10% target, where the share pays 0.7% of the goodput), 0.9-1.0× at 30-32%, 1.2-1.5× at 40-48%, and 1.9-2.1× from 61%. In the twelve settings the neural receiver adds 1.45-1.62 ms to the 99.9th percentile of the layer-1 latency of the

Table 15. Closed-loop link adaptation in 21 further settings: an Es/No that changes in every slot, two channels on one server, the low-correlation channel at two more Es/No values, and the urban micro channel. Cells as in Table 14. Rows without a cell count have four two-user cells.

| Setting, target | Gain | Antiphase | Fixed 10% | 30% share | 70% share | No cap |
|---|---|---|---|---|---|---|
| Slow change, 10% | +13.1% | 11.3k / −0.1% | 4.8k / −0.2% | 11.6k / −0.3% | 25.7k / −0.9% | 30.7k / −1.4% |
| Slow change, 1% | +29.0% | 36.9k / 0.0% | 5.7k / −0.1% | 18.8k / −0.2% | 44.2k / −0.4% | 50.2k / −1.0% |
| Slow change, 8 cells, 1% | +27.7% | 18.9k / −0.3% | 5.1k / −0.5% | 14.2k / −1.1% | 33.8k / −1.9% | 42.6k / −3.6% |
| Random change, 10% | +12.9% | 10.2k / 0.0% | 4.7k / −0.3% | 11.2k / −0.4% | 24.2k / −1.0% | 29.4k / −1.6% |
| Random change, 1% | +27.9% | 36.4k / −0.1% | 5.7k / −0.2% | 18.7k / −0.4% | 44.3k / −1.0% | 50.0k / −1.5% |
| Random change, 8 cells, 1% | +24.7% | 20.3k / −0.2% | 5.2k / −0.5% | 14.7k / −1.2% | 35.4k / −2.0% | 43.7k / −3.7% |
| Fast change, 10% | +13.8% | 12.0k / −0.1% | 4.8k / −0.2% | 11.6k / −0.3% | 25.7k / −0.8% | 30.3k / −1.2% |
| Fast change, 1% | +28.7% | 37.1k / 0.0% | 5.7k / −0.1% | 18.8k / −0.2% | 44.1k / −0.5% | 50.1k / −1.1% |
| Fast change, 8 cells, 1% | +26.5% | 19.3k / −0.1% | 5.1k / −0.5% | 14.2k / −0.7% | 33.6k / −1.5% | 42.3k / −3.3% |
| Mixed channels, 10% | +7.8% | 15.2k / 0.0% | 4.9k / −0.1% | 12.6k / −0.2% | 29.4k / −0.5% | 34.5k / −0.5% |
| Mixed channels, 1% | +18.5% | 41.1k / 0.0% | 5.7k / 0.0% | 20.1k / 0.0% | 46.6k / −0.2% | 51.5k / −0.3% |
| Mixed channels, 8 cells, 1% | +17.8% | 23.3k / −0.2% | 5.2k / −0.4% | 15.1k / −0.5% | 36.9k / −1.6% | 45.2k / −3.1% |
| Low correlation 4.5 dB, 10% | +5.5% | 11.0k / −0.2% | 4.6k / −0.1% | 11.3k / −0.3% | 24.7k / −0.6% | 30.3k / −0.9% |
| Low correlation 4.5 dB, 1% | +4.1% | 49.5k / 0.0% | 6.1k / 0.0% | 24.0k / 0.0% | 51.0k / 0.0% | 52.8k / 0.0% |
| Low correlation 4.5 dB, 8 cells, 1% | +4.6% | 40.6k / 0.0% | 5.7k / 0.0% | 20.1k / −0.1% | 46.0k / −0.1% | 51.0k / −0.2% |
| Low correlation 7.5 dB, 10% | +5.5% | 27.3k / 0.0% | 5.2k / 0.0% | 15.8k / 0.0% | 37.0k / −0.3% | 43.0k / −0.3% |
| Low correlation 7.5 dB, 1% | +6.8% | 48.6k / 0.0% | 6.0k / 0.0% | 23.4k / 0.0% | 50.5k / 0.0% | 52.8k / 0.0% |
| Low correlation 7.5 dB, 8 cells, 1% | +6.6% | 38.8k / 0.0% | 5.6k / 0.0% | 19.6k / 0.0% | 44.4k / −0.1% | 50.2k / −0.1% |
| Urban micro, 10% | +5.3% | 24.4k / 0.0% | 5.2k / 0.0% | 15.3k / 0.0% | 35.7k / −0.1% | 41.6k / −0.3% |
| Urban micro, 3% | +1.5% | 36.9k / 0.0% | 5.6k / 0.0% | 18.4k / 0.0% | 43.6k / 0.0% | 49.4k / −0.3% |
| Urban micro, 8 cells, 3% | +1.4% | 16.1k / −0.1% | 4.8k / −0.1% | 13.1k / −0.1% | 29.4k / −0.3% | 37.1k / −0.7% |

single-user cells, Antiphase adds 0.00-0.13 ms on top, the 30% share 0.23-0.34 ms, and low priority alone 0.60-0.89 ms.

**A Continuously Changing Channel, Mixed Channels, and a Third Channel.** We extend the settings in three directions (Table 15). First, the Es/No of a cell changes in every slot. We add white noise to slots generated at 20 dB: the noise variance of the generator is $1.267 \cdot 10^{-e/10}$ at an Es/No of $e$ dB for every MCS and channel, and the added variance sets the Es/No of each slot. Slots made in this way at 16 and 14 dB match generated slots within 2.9 percentage points of decoded TBs, for both receivers and at all seven MCS levels. The Es/No moves between 14 and 20 dB with a period of 5.12 s (slow, up to 3.7 dB/s) or 1.28 s (fast, up to 14.7 dB/s), or it follows a random process with a mean of 16.5 dB, a standard deviation of 1.75 dB, and a correlation time of 1 s. Antiphase loses 0.0-0.3% of the goodput in the nine settings, and the shares lose up to 3.7%; the order of the policies and the size of the losses are those of the fixed channel. At the 10% target the outer loop follows all three trajectories: its mean MCS rises by about two levels from the lowest to the highest Es/No, and 8.7-13.3% of the TBs need a retransmission at every Es/No. At the 1% target it does not follow the fast change, where its MCS moves by 0.4 levels and 2.4% of the TBs need a retransmission at the lowest Es/No; the comparison of the policies is the same. Second, half of the two-user cells of a server use the high-correlation channel at 16 dB and half the low-correlation channel at 6 dB. The loss of a share falls on the high-correlation cells: with eight two-user cells at the 1% target, the 70% share loses 1.6% on average, 3.1% in the high-correlation cells and 0.2% in the others, and low priority alone loses 6.4% and 0.2%. Antiphase loses 0.4% and 0.0%. Thus, a cap must be chosen for the worst cell of a server. Third, we run the low-correlation channel at 4.5 and 7.5 dB and the urban micro channel at 10 dB. The recovery path adds 4-9% on the low-correlation channel and 1-5% on the urban micro channel, and a lost recovery costs little there: the shares lose at most

Table 16. AI on a slice of the SMs of a GPU (a CUDA green context), with the outer loop (two seeds). Cells as in Table 14; settings by the number of two-user cells and the target. Slice $N$: Antiphase with the AI of a GPU on $N$ SMs while the neural receiver of that GPU runs. Always 28: the AI runs on 28 SMs at all times. Each block is one campaign with its own run of Antiphase.

*Neural receivers on all 108 SMs*

| Setting | Antiphase | Slice 12 | Slice 16 | Slice 28 | Always 28 | 30% share | 70% share |
|---|---|---|---|---|---|---|---|
| 4 cells, 10% | 11.8k / 0.0% | 8.8k / −0.1% | 11.7k / −0.2% | 13.6k / −0.3% | 8.8k / −0.2% | 11.4k / −0.2% | 25.7k / −0.7% |
| 4 cells, 1% | 33.7k / −0.1% | 28.5k / −0.1% | 31.7k / −0.2% | 33.4k / −0.5% | 13.2k / −0.2% | 17.7k / −0.3% | 41.9k / −0.8% |
| 8 cells, 1% | 16.8k / 0.0% | 13.9k / −1.0% | 16.9k / −0.9% | 20.6k / −1.9% | 10.2k / −1.2% | 13.9k / −1.9% | 34.0k / −3.6% |

*Neural receivers off the slice (on 92 or 80 SMs), or AI at normal priority*

| Setting | Antiphase | Slice 16, off | Slice 28, off | Always 28, normal | Always 28, off |
|---|---|---|---|---|---|
| 4 cells, 10% | 11.6k / +0.1% | 12.2k / −0.1% | 18.0k / −0.5% | 11.6k / −0.4% | 12.8k / −0.5% |
| 4 cells, 1% | 33.5k / 0.0% | 31.7k / −0.4% | 36.5k / −0.9% | 15.4k / −0.7% | 15.6k / −0.7% |
| 8 cells, 1% | 17.2k / −0.7% | 17.8k / −1.7% | 25.7k / −3.3% | 13.4k / −3.2% | 13.6k / −3.3% |

0.9% and 0.7%, and Antiphase at most 0.2% and 0.1%. On the urban micro channel the conventional receiver fails 1.5% of the TBs at the lowest MCS; we use a 3% target, at which the outer loop stays at the lowest MCS. Across the three channels the gain of the recovery path and the goodput that a share pays move together: 11-40% and up to 8.2%, 4-9% and up to 0.9%, and 1-5% and up to 0.7%. With the 21 settings above, Antiphase loses at most 0.5% of the goodput in 42 settings with one configuration, and at most 0.3% in 41 of them (eight two-user cells at the 3% target give 0.20% and 0.32% in two measurements). We measured the setting with the largest loss, eight two-user cells at the 1% target, with 33 seeds in 43 runs. The runs give 0.34% on average with a standard error of 0.05 points. Grouped by seed, the mean is 0.35% with a 95% interval of 0.25-0.44%, below 0.5%; with the first nine seeds alone the interval was 0.29-0.57%. A single run is between a gain of 0.25% and a loss of 1.02%, so the 0.5% above is a statement about the mean of a setting and its interval and not about every run. The 30% share loses 2.25% in its 14 runs of this setting, more than six times as much, and serves less AI than Antiphase.

**The Assumed Success of a Retransmission.** The runs do not send retransmissions: every slot carries a new TB, and the goodput counts a retransmission as one uplink slot of the user that succeeds with probability 0.95. We recomputed the goodput of every seed with a success of 0.80, 0.90, and 1.00 and with no gain from combining. The loss of every policy against the server without AI is the same to two decimals, for example 0.31% for Antiphase and 8.01% for low priority without a cap with eight two-user cells at the 1% target. The outer loop holds the share of TBs that need a retransmission at the target for every policy (1.00% or 10.00%), so the policies differ in the MCS at which the loop settles and not in the number of retransmissions, and the value of one retransmission cancels in the comparison.

**An SM Slice Does Not Replace the Rule.** A GPU can also be divided in space. A CUDA green context gives a process a set of the SMs of a GPU; Weaver [34] divides a GPU between LDPC decoding and AI training in this way. We build green contexts into the AI workers and the neural receiver processes and measure two uses (Table 16). First, as a baseline, the AI always runs on 28 of the 108 SMs: with low priority, with normal priority, or with the neural receivers on the other 80 SMs. In all nine cases this loses more goodput than Antiphase in the same campaign, 0.2-3.3%, and it serves 0.4-1.1× the AI, because the AI stays on 26% of the GPU when the neural receivers are idle. Second, as an option of Antiphase, the AI of a GPU moves to a slice of 12, 16, or 28 SMs while the neural receiver of that GPU runs, and it is not stopped. With 28 SMs the option serves 1.0-1.2×

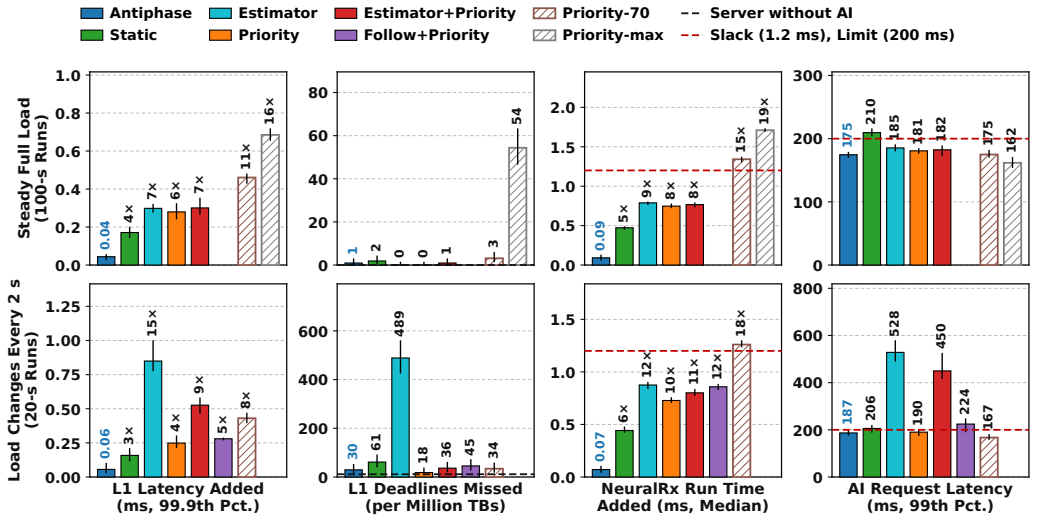



Fig. 15. What AI adds to the latency of the radio work, and the latency of the AI requests (16 cells, four GPUs, five seeds; lines: range over seeds, for the misses the 95% interval). Top: runs of 100 s without their first 20 s. Bottom: runs of 20 s without their first 5 s. L1 latency: samples arrive until the conventional result. The number on a baseline is how many times more it adds than Antiphase.

the AI of the default and loses 0.3-1.9% of the goodput; with the neural receivers on the other 80 SMs it serves 1.1-1.6× and loses 0.5-3.3%. Slices of 12 and 16 SMs serve no more AI than the default. The loss follows the run time of the neural receiver, whatever divides the GPU. The policies with a median run of 6.4-6.6 ms (Antiphase) lose 0.0-0.7%, those with 6.8-7.2 ms lose 0.1-1.9%, and those with 7.4-7.9 ms lose 0.4-3.6%, shares and slices alike. This is the path of the model (Section 4): a run has 1.2 ms of slack, and the neural receiver alone takes 0.84 ms longer on 80 SMs than on 108. A slice isolates the AI, and the neural receiver needs the SMs. The AI gains little on a slice next to a running neural receiver: a piece takes 7.1× its bound on the whole GPU on 28 shared SMs and 4.0× on 28 SMs of its own. In runs of 100 s, the option with 28 SMs adds 0.23 ms to the 99.9th percentile of the layer-1 latency, and the default adds 0.06 ms. The fixed split with the neural receivers on 80 SMs lowers this percentile to 0.25 ms below the server without AI, because the conventional receivers no longer share 28 SMs with the neural receiver, and it loses 8.3% of the recoveries. The default of Antiphase has no slice.

### 6.7 Protection and latency

**Takeaway.** In steady state Antiphase adds 0.04-0.06 ms to the 99.9th-percentile L1 latency and 0.07-0.09 ms to the run of the neural receiver, 2.8-21× and 5.2-19× less than the baselines. After the start of a run, four to eight of a million TBs pass the layer-1 deadline with Antiphase against two without AI, and its AI requests finish within their limit at the 99th percentile.

Figure 15 and Table 17 give the metrics of a scheduler: what AI adds to the latency of the radio work, the deadlines missed, the latency of the AI requests, and the use of the GPU time. The values at full load are from runs of 100 s (40,000 uplink slots per cell), measured in one job with the policies in turn.

**Three Levels.** The latency of the radio work on an AI-RAN server is not zero at any level, and we separate what the neural receiver adds from what AI adds. Table 19 compares the single-user

Table 17. Scheduling metrics at a steady full load (16 cells, four GPUs; five runs of 100 s per policy without their first 20 s, 3.2 million TBs per policy). L1 latency and neural receiver run time are in ms; AI latency is from the arrival of a request to the end of its prefill, in ms. cuPHY alone: a server without the neural receiver and without AI.

| Policy | AI served (k tokens/s) | Recoveries kept (%) | L1 latency p50 / p99.9 | TBs past L1 deadline | NeuralRx run p50 / p99 | AI latency p50 / p99 | Requests rejected (%) | GPU idle (%) |
|---|---|---|---|---|---|---|---|---|
| cuPHY alone | – | – | 1.38 / 1.49 | 0 | – | – | – | – |
| No AI | 0.0 | 100.0 | 1.39 / 3.01 | 0 | 6.28 / 6.57 | – | – | 26.4 |
| Antiphase | 34.1 | 99.9 | 1.81 / 3.06 | 3 | 6.38 / 6.82 | 77 / 175 | 16.4 | 1.0 |
| *Static* | 5.6 | 99.3 | 1.47 / 3.18 | 6 | 6.76 / 7.05 | 120 / 210 | 64.9 | 2.1 |
| *Estimator* | 11.0 | 98.7 | 1.54 / 3.31 | 0 | 7.07 / 7.36 | 84 / 185 | 46.8 | 1.4 |
| *Priority* | 17.6 | 98.8 | 1.61 / 3.29 | 0 | 7.03 / 7.29 | 79 / 181 | 33.9 | 1.5 |
| *Estimator+Priority* | 17.6 | 98.7 | 1.62 / 3.31 | 3 | 7.05 / 7.31 | 77 / 182 | 33.8 | 1.4 |
| *Priority-70* | 42.6 | 94.3 | 1.80 / 3.47 | 10 | 7.63 / 7.98 | 73 / 175 | 9.2 | 1.1 |
| *Priority-max* | 48.9 | 93.7 | 1.90 / 3.70 | 174 | 8.00 / 8.53 | 61 / 162 | 4.0 | 1.7 |

Table 18. TBs past the layer-1 deadline by the time since the start of a run (runs of 100 s; five per policy with 16 cells and with one GPU, three with 20 and 48 cells).

| Policy | First 5 s | 5-10 s | 10-20 s | After 20 s |
|---|---|---|---|---|
| *16 cells, four GPUs (3.2 million TBs after 20 s)* | | | | |
| No AI | 125 | 14 | 0 | 0 |
| Antiphase | 113 | 7 | 8 | 3 |
| *Static* | 74 | 11 | 2 | 6 |
| *Estimator* | 40 | 19 | 7 | 0 |
| *Priority* | 101 | 11 | 0 | 0 |
| *Estimator+Priority* | 74 | 8 | 0 | 3 |
| *Priority-70* | 185 | 13 | 13 | 10 |
| *Priority-max* | 167 | 41 | 19 | 174 |
| *4 cells, one GPU (0.8 million TBs after 20 s)* | | | | |
| No AI | 23 | 0 | 0 | 0 |
| Antiphase | 64 | 2 | 1 | 0 |
| *Static* | 48 | 2 | 0 | 46 |
| *Priority* | 41 | 0 | 0 | 21 |
| *20 cells, four GPUs (three runs, 2.3 million TBs after 20 s)* | | | | |
| No AI | 88 | 0 | 0 | 2 |
| Antiphase | 99 | 0 | 2 | 3 |
| *Static* | 54 | 4 | 0 | 0 |
| *Priority* | 70 | 5 | 0 | 1 |
| *48 cells at one third load, four GPUs (three runs, 1.9 million TBs after 20 s)* | | | | |
| No AI | 30 | 0 | 0 | 17 |
| Antiphase | 70 | 0 | 0 | 17 |
| *Static* | 42 | 10 | 3 | 26 |
| *Priority* | 64 | 11 | 3 | 14 |

cells, whose workload does not change, on a server with the conventional receiver alone, with the neural receiver, and with AI. The neural receiver adds 1.53-1.59 ms to the 99.9th percentile of the layer-1 latency, because the conventional receiver of a GPU slows while the neural receiver of that GPU runs. This takes 62-64% of the room to the deadline and leaves 0.90-0.95 ms. It is the price of the recovery path, which raises the goodput by 11-31% (Section 6.6). AI must fit into the room that is left. Antiphase adds 0.03-0.07 ms. *Static* adds 0.17-0.21 ms, *Priority* adds 0.30-0.36 ms, and

Table 19. Layer-1 latency of the single-user cells, whose workload is the same at every level (runs with the outer loop of Section 6.6; 20 s per run without the first 5 s; range over four settings). Room: time left to the 4.0 ms deadline at the 99.9th percentile.

| Server | Median (ms) | 99.9th percentile (ms) | Room (ms) |
|---|---|---|---|
| Conventional receiver alone | 1.39-1.41 | 1.50-1.52 | 2.48-2.50 |
| + neural receiver, no AI | 1.40-2.09 | 3.05-3.10 | 0.90-0.95 |
| + AI, Antiphase | 1.88-2.07 | 3.12-3.14 | 0.86-0.88 |
| + AI, *Static* | 1.50-2.17 | 3.22-3.31 | 0.69-0.78 |
| + AI, *Priority* | 1.70-2.31 | 3.37-3.41 | 0.59-0.63 |
| + AI, *Priority-70* | 1.85-2.41 | 3.48-3.56 | 0.44-0.52 |
| + AI, *Priority-max* | 1.95-2.54 | 3.73-4.00 | 0.00-0.27 |

*Priority-max* adds 0.68-0.90 ms, which leaves 0.00-0.27 ms. We repeated the 10% target, where the neural receivers run for 68% of the time, in runs of 100 s (three seeds, 1.15 million TBs of single-user cells per level after the first 20 s). No TB passes the layer-1 deadline with the conventional receiver alone or with the neural receiver, and 3 do with Antiphase. *Priority* serves the same AI as Antiphase in this setting (11.5k tokens/s); it adds 0.29 ms to the 99.9th percentile against 0.03 ms, loses 4.2% of the recoveries against 0.9%, and 11 TBs pass the deadline. With *Priority-max* the 99.99th percentile is 4.20 ms and 470 TBs pass the deadline. On the low-correlation channel at 6 dB the levels are the same (three runs of 100 s): 1.49 ms with the conventional receiver alone, 3.13 ms with the neural receiver, and 3.13 ms with Antiphase, which serves 21.1k tokens/s with 4 late TBs of 1.15 million. *Priority* serves 14.3k and leaves 0.61 ms of room against 0.87 ms, and *Priority-max* has 505 late TBs and a 99.99th percentile of 4.21 ms. At a steady full load with a fixed MCS (first rows of Table 17), the conventional receiver alone has 1.49 ms at the 99.9th percentile and the neural receiver raises it to 3.01 ms. In the rest of this section, the server without AI is the second level: it runs the neural receiver.

**Start of a Run.** The processes of a run start cold, and its first seconds are not the steady state. Table 18 counts the TBs past the layer-1 deadline by the time since the start. Without AI, 139 TBs are late in five runs, and all of them are in the first 10 s. With Antiphase, 128 of 131 are in the first 20 s. The runs of 10-40 s in the rest of this section include this start: their layer-1 misses of 0.02-0.04% occur with and without AI, and they compare the policies at an equal run length, not in steady state.

**L1 Latency.** In steady state a TB is decoded in 3.01 ms at the 99.9th percentile without AI, 0.99 ms before the layer-1 deadline. Antiphase adds 0.04 ms. *Static* adds 0.17 ms, *Priority* and the two estimator policies 0.28-0.30 ms, *Priority-70* 0.46 ms, and *Priority-max* 0.69 ms. At the 99.99th percentile Antiphase equals the server without AI (3.18 ms), and the baselines are at 3.34-3.94 ms. After the first 20 s, no TB of 3.2 million is late without AI, three are late with Antiphase, at most ten with the baselines that cap the share, and 174 with *Priority-max*. With one GPU, no TB of 0.8 million is late after the first 20 s without AI or with Antiphase, and *Static* and *Priority* miss the deadline for 46 and 21 TBs. With 20 cells, Antiphase has 3 late TBs of 2.3 million against 2 without AI, and with 48 cells 17 of 1.9 million against 17. In these 16 runs the late TBs of Antiphase after the start cannot be told from those of the server without AI. With more runs they can. We repeated the runs of 100 s in later campaigns, and over 36 runs each (20.6 million TBs; 16, 20, 32, and 48 cells and one GPU) the server without AI has 44 late TBs after the first 20 s and Antiphase has 155, of which 74 fall in 0.1 s of one run on all four GPUs. This is two per million against four to eight. The late TBs come in rare events of two to six TBs at one instant: 11 events without AI and 25 with Antiphase, 0.3 and 0.7 per run (two-sided $p = 0.03$). In the same jobs as the first runs, *Priority-max*

Table 20. Rules changed one at a time (16 cells at full load, 53k tokens/s offered; five seeds, rows marked * two). Without AI: layer-1 misses 0.032%, lost candidates 1.6%, three-slot runs 10%.

| Configuration | AI tokens/s | Recoveries kept | Layer-1 misses | Lost candidates | Three-slot runs |
|---|---|---|---|---|---|
| Low-priority AI, no grants | 49.0k | 93.7% | 0.044% | 6.4% | 88% |
| Grants without stops, AI next to a neural receiver while three are free* | 39.5k | 97.8% | 0.025% | 3.2% | 52% |
| Stops, AI next to a neural receiver while one is free* | 44.0k | 95.1% | 0.066% | 5.2% | 81% |
| Stops, while two are free* | 42.2k | 96.6% | 0.051% | 4.1% | 64% |
| Stops, while three are free | 39.8k | 99.3% | 0.038% | 2.4% | 35% |
| Stops, no AI next to a neural receiver | 37.6k | 100.0% | 0.061% | 1.8% | 14% |
| + largest units not next to the conventional receivers (Antiphase) | 34.1k | 99.9% | 0.036% | 1.7% | 11% |

has 54 late TBs per million with 16 cells, and *Static* and *Priority* 57 and 26 per million with one GPU. When the load changes, the two estimator policies add 0.53-0.87 ms, because their share is one second behind the load, and *Estimator* misses the deadline for 0.05% of the TBs; Antiphase adds 0.04-0.06 ms. Antiphase raises the median L1 latency from 1.39 to 1.81 ms, as much as *Priority-70*, because the two smaller unit classes run next to the conventional receivers; with only the smallest class next to them the median rises by 0.20 ms (Section 6.8). The tail does not move.

**Neural Receiver.** A run of the neural receiver takes 6.28 ms at the median without AI and 6.38 ms with Antiphase. The baselines add 0.44-1.71 ms. *Priority-70* and *Priority-max* are beyond the slack of 1.2 ms (Equation 3), which is where they lose recoveries.

**AI Latency.** The AI requests have a limit of 200 ms. With Antiphase the 99th percentile of the request latency is 175-187 ms, and 99.0-99.8% of the placed requests finish within the limit. At a steady full load the baselines are at 162-210 ms: *Request Dispatcher* places a request only if a GPU can finish it in time, and a policy with less GPU time rejects more requests, 16% with Antiphase, 34% with *Priority*, and 65% with *Static*. When the load changes, the two estimator policies take 388-528 ms. The rejections follow the offered load: at 27k tokens/s offered, Antiphase rejects 0.6% of the requests and has a 99th-percentile latency of 115 ms (Section 6.11).

**GPU Time.** Without AI, the GPUs have no work on them for 26.4% of the time at full load and for 37-43% under the two changing loads. With Antiphase they have no work for 1.0-4.0%. Every policy with AI keeps the GPUs busy for 89-99% of the time; the policies differ in what the busy time does to the radio work. An AI piece is on a GPU next to a running neural receiver for 1.4-2.4% of the time with Antiphase, the unit that finishes after a stop, and for 19-38% with the baselines.

### 6.8 What each rule adds

**Takeaway.** Stoppable AI pieces and no AI next to a running neural receiver raise the recoveries kept from 97.8% to 100.0%. Admitting AI by the deadline of each neural receiver run keeps every run inside its deadline and keeps 93.8%. Keeping the largest AI units away from the conventional receivers is a conservative setting from a calibration: it lowers the layer-1 misses of runs of 10 s from 0.061% to 0.036%, changes neither the goodput nor the late TBs in steady state, and costs 9-12% of the AI served.

Table 20 changes the rules one at a time.

**Stops.** Without stops, AI is still running for 43% of the neural receiver time during which the rule wants it off, the neural receiver takes 7.04 ms at the median, and 52% of the runs take three slots.

Table 21. Configurations one rule away from Antiphase under rate control (three seeds): goodput of the two-user cells against the server without AI / AI served (tokens/s) / three-slot runs.

| Two-user cells, target | Antiphase | Stop only | Own deadline only | Low priority, no cap |
|---|---|---|---|---|
| 4, 10% | −0.00% / 11.5k / 25% | −0.01% / 11.5k / 27% | −0.65% / 22.8k / 91% | −0.96% / 30.3k / 94% |
| 4, 1% | −0.10% / 33.7k / 19% | −0.07% / 37.3k / 22% | −0.89% / 41.0k / 96% | −1.60% / 48.7k / 89% |
| 8, 1% | −0.50% / 17.2k / 34% | −0.24% / 17.7k / 35% | −3.46% / 32.0k / 97% | −7.69% / 45.0k / 94% |

With stops the figure is 6-7%, the time until the group on the GPU ends, and the neural receiver takes 6.38 ms against 6.28 ms alone.

**AI Next to a Neural Receiver.** Allowing AI next to a running neural receiver while $R$ neural receivers of the server are free trades recoveries for AI. From no AI next to a neural receiver to $R = 3$, 2, and 1, the three-slot runs grow from 14% to 35%, 64%, and 81%, the lost candidates from 1.8% to 2.4%, 4.1%, and 5.2%, and the AI served from 37.6k to 44.0k tokens/s. Antiphase uses no AI next to a neural receiver: it keeps the recoveries of the run without AI and has no value to set for the number of GPUs. The event simulation gives a difference of 0.1 points between this rule and $R = 3$ and the measurement gives 0.6, a difference inside the model error.

**Unit Sizes Next to the Conventional Receivers.** With every size class allowed next to the conventional receivers, the layer-1 misses rise to 0.061%: next to 1,024-token units the conventional receivers of four cells finish in 4.05 ms at the 99.9th percentile. Keeping the largest class away from them returns the misses to 0.036% and costs 9% of the AI served. Two variants that choose the classes by the number of active cells of the GPU did not help. Allowing only the smallest class when five cells are active and none above, as a calibration with five cells per GPU suggests, cut the AI served by 28-51% with 20, 32, and 48 cells and left the layer-1 misses unchanged: the calibration runs the neural receiver and AI next to the conventional receivers at once, which Antiphase never does. Allowing the largest class while at most two or three cells are active served 0-19% less AI under a changing load, because a request that has picked large chunks waits when more cells become active. The size classes next to the conventional receivers also set the median L1 latency: it is 1.40 ms without AI, 1.60 ms with only the smallest class next to them (24.2k tokens/s served), 1.82 ms with the two smaller classes (33.9k), and 1.83 ms with every class (37.3k). The 99.9th percentile is the same without AI and with one or two classes (3.07-3.12 ms) and 3.51 ms with every class. Antiphase allows two classes because the layer-1 decode has a deadline; a deployment that needs a lower median changes this parameter and no rule.

**One Rule Away.** Table 21 runs two configurations that differ from Antiphase in one rule, under rate control. *Stop only* drops the rule on unit sizes: the AI of a GPU stops when its neural receiver starts, and every unit size may run next to the conventional receivers. *Own deadline only* replaces the stop by a deadline check: AI runs next to a neural receiver as long as that run still meets its own recovery deadline. At a fixed MCS and full load (five seeds), own deadline only serves 41.3k tokens/s and keeps 93.8% of the recoveries. All of its neural receiver runs end inside the recovery deadline, 92.7% of them hold a third slot (10.0% without AI), and 6.6% of the candidates find no free neural receiver. Under rate control it loses 0.65-3.46% of the goodput. This is the finding of §4 measured directly: protecting the deadline of every neural receiver run does not protect the recoveries. Stop only has the goodput of Antiphase in all three settings (the ranges over the seeds overlap) and serves 0-11% more AI: the goodput is kept by the recovery path and the stop, not by the rule on unit sizes.

**What the Rule on Unit Sizes Protects.** The rule on unit sizes is meant to protect the layer-1 deadline of the conventional receivers. Layer-1 misses concentrate at the start of a run (§6.7), so

Table 22. The rule on unit sizes in runs of 100 s without their first 20 s: no AI / Antiphase/ stop only.

| Setting | Seeds | TBs | Late TBs | L1 99.99th pct. (ms) | AI served, Antiphase / stop only |
|---|---|---|---|---|---|
| 16 cells | 3 | 1.92M | 1 / 0 / 0 | 3.20 / 3.18 / 3.25 | 34.0k / 37.8k |
| 20 cells | 8 | 6.14M | 2 / 90 / 10 | 3.30 / 3.40 / 3.42 | 31.9k / 35.5k |
| 32 cells, half load | 3 | 1.92M | 0 / 10 / 2 | 3.38 / 3.39 / 3.41 | 33.6k / 37.5k |
| 48 cells, one third load | 3 | 1.92M | 18 / 22 / 23 | 3.49 / 3.50 / 3.54 | 33.0k / 36.9k |
| One GPU, 4 cells | 3 | 0.48M | 4 / 10 / 0 | 3.09 / 3.13 / 3.09 | 8.2k / 8.9k |

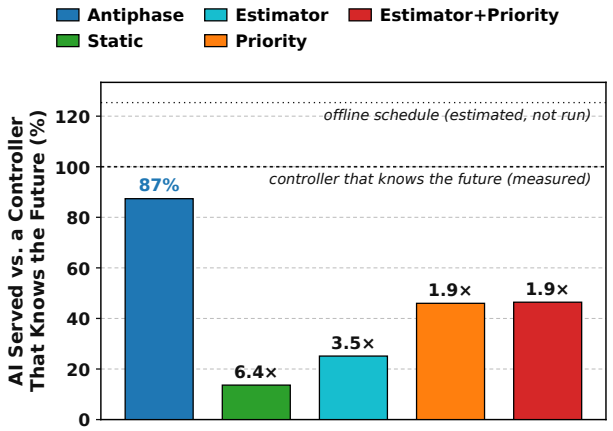



Fig. 16. AI served by each policy against a clairvoyant controller that was run (100%; it knows the candidates of every slot in advance and keeps one neural receiver free beyond the known demand), at a steady full load (16 cells, four GPUs). The upper line is the estimate for an offline schedule, which is not run. The number on a baseline is the factor of Antiphase over it.

we compare Antiphase and stop only in runs of 100 s without their first 20 s (Table 22). In none of the five settings does the rule lower the late TBs. They come in rare events of two to six TBs at one instant under both configurations; 68 of the 90 late TBs of Antiphase with 20 cells fall in 0.1 s of one run on all four GPUs. With the first three seeds at 20 cells, stop only had six late TBs and Antiphase none; with eight seeds the order reversed, so that difference was noise. The tails differ by 0.00-0.04 ms at the 99.9th percentile and by −0.04 to 0.07 ms at the 99.99th. The rule costs 9-12% of the AI served. Its basis is the calibration above, in which the conventional receivers of four cells finish in 4.05 ms at the 99.9th percentile next to 1,024-token units that run all the time; in the runs such units are not next to the conventional receivers all the time. We keep the rule as a conservative default, every headline number of this paper is measured with it, and a deployment can drop it for 9-12% more AI at the same goodput.

### 6.9 Distance to a clairvoyant schedule

**Takeaway.** A controller that knows the failures of every slot in advance, which we ran, serves 15% more AI than Antiphase at full load while it keeps 99.5% of the recoveries: Antiphase serves 87% of it (76-95% in six settings), which is 1.9-6.4× the existing mechanisms that keep the recoveries.

Table 23. Share of the GPU time in which AI may run with no recovery lost, computed on the runs without AI, and AI served. The clairvoyant AI served scales the measured AI served of Antiphase by the AI time and is at most what *Priority-max* serves (full load) or the offered load.

| Setting | Neural receiver busy | AI time rule, ideal | AI time Antiphase | AI time clairvoyant | AI served (tokens/s) Antiphase | AI served (tokens/s) clairvoyant | Antiphase / clairvoyant |
|---|---|---|---|---|---|---|---|
| 16 cells, full load | 36% | 64% | 62% | 96.5% | 34.1k | 49.0k | 70% |
| 16 cells, load alternates | 27% | 73% | 69% | 98.1% | 41.4k | 53.0k | 78% |
| 16 cells, random steps | 22% | 78% | 72% | 98.9% | 44.7k | 53.0k | 84% |
| Two two-user cells | 18% | 82% | 77% | 99.6% | 46.6k | 53.0k | 88% |
| Eight two-user cells | 67% | 33% | 33% | 76.1% | 11.8k | 27.3k | 43% |
| Two GPUs, 8 cells | 34% | 66% | 61% | 93.8% | 16.6k | 25.6k | 65% |
| One GPU, 4 cells | 30% | 70% | 59% | 94.6% | 8.5k | 13.2k | 64% |
| 32 cells, half load | 37% | 63% | 62% | 97.5% | 33.4k | 52.6k | 63% |

Table 24. Antiphase against a clairvoyant controller that was run: AI served (tokens/s) at the share of recoveries kept. Margin: neural receivers kept free beyond the known demand. Five seeds at full load, two elsewhere.

| Setting | Antiphase | Clairvoyant controller margin 1 | Clairvoyant controller margin 0 | Antiphase / margin 1 |
|---|---|---|---|---|
| 16 cells, full load | 33.9k at 99.9% | 39.1k at 99.5% | 39.6k at 98.3% | 87% |
| 16 cells, load alternates | 41.1k at 99.9% | 44.6k at 99.7% | 45.0k at 98.7% | 92% |
| 16 cells, random steps | 44.2k at 99.8% | 47.1k at 99.8% | 47.4k at 99.1% | 94% |
| Two two-user cells | 46.7k at 99.7% | 49.0k at 99.3% | 48.8k at 99.3% | 95% |
| Six two-user cells | 19.9k at 99.7% | 25.0k at 99.7% | 27.5k at 98.0% | 80% |
| Eight two-user cells | 11.6k at 99.5% | 15.3k at 99.2% | 17.5k at 97.0% | 76% |

The comparisons above are with other online policies. We compare Antiphase with knowledge of the future in two ways: a clairvoyant controller that we ran (Table 24) and an estimate for an offline schedule (Table 23).

**A Clairvoyant Controller That Was Run.** The candidates of every slot are fixed by the data set, the cell, and the seed, so a run without AI gives a table of the candidates per slot for every run of the same seed. The controller reads this table. For a neural receiver run of slot $k$ it asks whether the candidates of slot $k + 2$ will need that receiver: the runs next to which AI may continue are at most the number of neural receivers minus the runs of slot $k + 1$, the candidates of slot $k + 2$, and a margin. Next to those runs the AI keeps running, as long as the run meets its own recovery deadline, and the run then holds a third slot; next to the other runs the AI stops as in Antiphase. The rule on unit sizes and the stoppable pieces are those of Antiphase. The controller knows neither the run lengths nor the AI arrivals. It is a baseline, not a setting of Antiphase.

With one receiver kept free, the clairvoyant controller serves 39.1k tokens/s at full load and keeps 99.5% of the recoveries; AI runs next to 82% of its neural receiver runs. Antiphase serves 87% of this, and 76-95% in the six settings: knowing the future is worth 5-32% more AI. Without the margin the controller serves up to 14% more and keeps 97.0-99.3% of the recoveries, because one run in ten holds a third slot even without AI next to it. The clairvoyant controller also pays in the layer-1 tail: its 99.9th percentile is 3.52-3.73 ms against 3.04-3.19 ms for Antiphase and 3.01-3.15 ms without AI, because AI next to a neural receiver also delays the conventional receivers. The existing mechanisms that keep the recoveries serve 14-46% of the controller at full load (Figure 16). A variant that also runs AI next to the needed runs, while their predicted end stays before their reuse time, serves 3-4% more AI (40.2k-40.5k tokens/s) and keeps 95.0-96.0% of the recoveries, so the reference

that keeps the recoveries is the controller above. The controller is one rule and not the optimum; the estimate below is a reference on that side.

**An Estimate for an Offline Schedule.** Table 23 is computed on the runs without AI and is not run. The schedule knows every later candidate. It keeps the start time of every neural receiver run and loses no candidate. It runs AI next to a neural receiver for as long as the run still ends before the next run on that receiver starts; a run next to AI advances at 0.786 of its speed alone. For fixed start times this schedule is optimal: assigning each run to the receiver whose stretched run ends first minimizes the time that AI must be off. Its AI served is an estimate scaled from the AI time, capped at the measured AI of low priority without a cap; we use it as a reference and not as a proven upper bound.

**The Rule Is Implemented Tightly.** At full load the neural receivers run for 36% of the GPU time. The rule of Antiphase allows AI for the other 64%, and AI is on the GPU for 62%: stops and grants cost two points.

**What the Estimate Says.** The offline schedule turns AI off for 3.5% of the time: only 13% of the runs must be cut for a later candidate. Antiphase serves 70% of the estimated AI at full load, 78-88% at lower loads, and 43% with eight two-user cells. The controller that was run reaches 80% of the estimate at full load (39.1k of 49.0k tokens/s), so the estimate is on the high side. The existing mechanisms that keep at least 98.8% of the recoveries serve 11-37% at full load, and allowing AI next to a neural receiver while three are free serves 81% and loses 0.7% of the recoveries. The gap is one of information. Which neural receiver is needed two slots later is known only when that slot is decoded, 5.0 ms into the run; AI next to the run until then has delayed it by 1.07 ms of the 1.2 ms slack, less than the stop latency and the spread of the run length leave. A rule that spends the slack lost 4.0% of the recoveries in our runs. Closing the gap needs a prediction of the failures two slots ahead; the correlation between the failures of consecutive slots (0.19-0.26 against a mean of 0.145) is too weak for it. The next paragraph measures what such a prediction would have to deliver.

**What a Prediction Would Have to Deliver.** The controller above reads an exact table. A predictor of the failures errs, so we ran the controller with errors in its table (Figure 17; headline setting, three seeds). In a table with misses, each true candidate is kept with a given probability; in a table with false alarms, each slot of a two-user cell without a candidate is shown as one with a given probability. The two errors cost different things. A missed candidate lets AI run next to a neural receiver that is needed, and the recovery is lost: the controller keeps 99.3% of the recoveries with 10% of the candidates missing, 98.1% with 25%, and 95.8% with 50%. A false alarm only stops AI without need: with half of the other cell-slots shown as candidates, three of four entries of the table are wrong, and the controller keeps 99.7% and serves 1.04× the AI of Antiphase. A predictor therefore needs to find nine of ten candidates and may raise many false alarms; with no misses and 10-25% false alarms the controller serves 8-12% more AI than Antiphase. The prize is bounded: an exact table is worth 15% more AI, and every table costs 0.4-0.7 ms in the layer-1 tail (3.50-3.75 ms against 3.09 ms). With an empty table the controller serves 40.9k tokens/s at 93.5%, close to the baseline that only checks the deadline of the neural receiver (41.3k at 93.8%, §6.8).

### 6.10 Neural receiver demand and server size

**Takeaway.** From one to four GPUs, Antiphase serves 6.4-7.8× the AI of *Static* and 1.9-2.2× that of *Priority* (Figure 18a). With more two-user cells, the neural receivers run for more of the time and the factor falls: from 8.2× to 2.7× over *Static* and from 2.1× to 1.0× over *Priority*, which loses 4.0% of the recoveries with eight two-user cells where Antiphase loses 0.4% (Figure 18c).

**Demand.** Table 25 raises the neural receiver demand by making more cells two-user cells. With two and four GPUs, Antiphase keeps at least 99.6% of the recoveries in all six settings and serves 2.7-8.2× the AI of *Static* and 1.0-2.1× that of the 30% share with low-priority AI. Antiphase gives

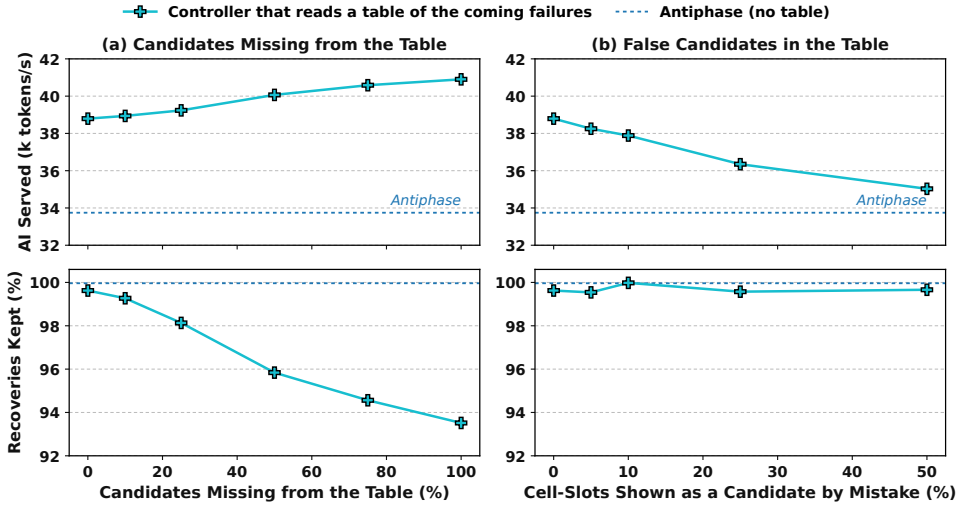



Fig. 17. The clairvoyant controller with errors in its table: AI served (top) and recoveries kept (bottom) with (a) candidates missing from the table and (b) cell-slots shown as candidates by mistake. Dotted: Antiphase, which has no table.

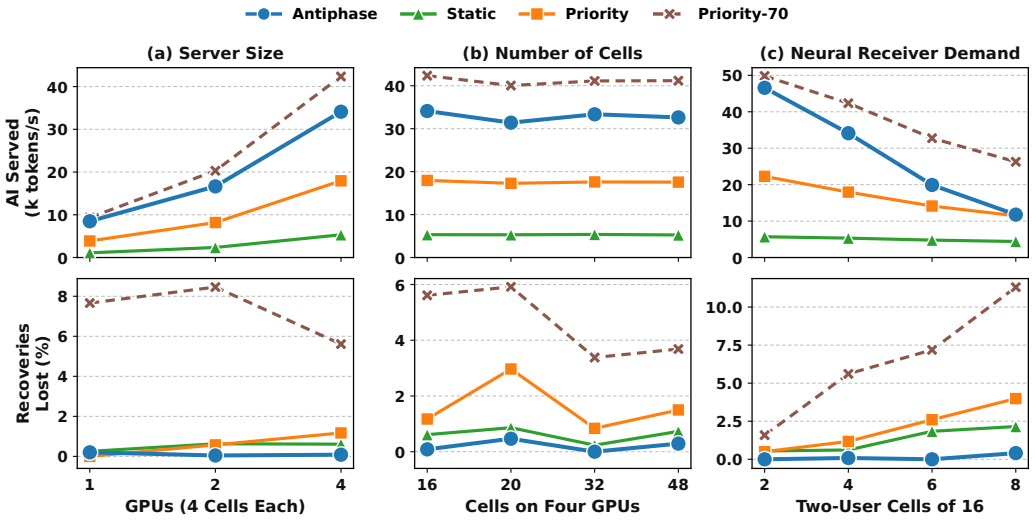



Fig. 18. AI served (top) and recoveries lost (bottom) at full load for (a) one, two, and four GPUs with one two-user cell per GPU, (b) 16-48 cells on four GPUs, and (c) two to eight two-user cells of sixteen. *Priority-70* (dashed) serves more AI and loses 1.6-11.3% of the recoveries.

GPU time back as the demand grows: the neural receivers run for a larger part of the time, and the AI served falls from 46.6k tokens/s with two two-user cells to 11.8k with eight. The 70% share with low-priority AI serves 1.1-2.2× the AI of Antiphase and loses 1.6-11.7% of the recoveries.

Table 25. Neural receiver demand and server size at full load, 32 requests/s offered per GPU (two seeds; 16 cells with four two-user cells and the two settings with one GPU: five). Each cell shows AI tokens/s and recoveries kept.

| GPUs, cells, two-user cells | Recovered TBs/s, no AI | Antiphase | Fixed 10% | 30%, low priority | 70%, low priority |
|---|---|---|---|---|---|
| 4, 16, 2 | 101 | 46.6k, 100.0% | 5.7k, 99.5% | 22.3k, 99.5% | 49.8k, 98.4% |
| 4, 16, 4 | 187 | 34.1k, 99.9% | 5.3k, 99.4% | 18.0k, 98.8% | 42.4k, 94.4% |
| 4, 16, 6 | 283 | 19.9k, 100.0% | 4.8k, 98.2% | 14.1k, 97.4% | 32.8k, 92.8% |
| 4, 16, 8 | 359 | 11.8k, 99.6% | 4.4k, 97.8% | 11.5k, 96.0% | 26.3k, 88.7% |
| 2, 8, 2 | 95 | 16.6k, 100.0% | 2.4k, 99.4% | 8.2k, 99.4% | 20.3k, 91.5% |
| 2, 8, 4 | 154 | 6.9k, 99.9% | 2.0k, 99.2% | 5.8k, 96.6% | 14.2k, 88.3% |
| 1, 4, 1 | 39 | 8.5k, 99.8% | 1.1k, 99.7% | 3.9k, 100.0% | 9.3k, 92.3% |
| 1, 4, 2 | 64 | 4.5k, 97.7% | 1.0k, 97.5% | 2.9k, 96.8% | 8.1k, 84.1% |

Table 26. More cells on four GPUs, 53k tokens/s offered (four seeds). Each cell shows AI tokens/s, recoveries kept, and layer-1 misses.

| Policy | 20 cells, full load | 32 cells, half load | 48 cells, one third load |
|---|---|---|---|
| Antiphase | 31.4k, 99.5%, 0.044% | 33.4k, 100.0%, 0.041% | 32.6k, 99.7%, 0.130% |
| Fixed 10% | 5.3k, 99.1%, 0.037% | 5.4k, 99.8%, 0.038% | 5.3k, 99.3%, 0.125% |
| Fixed 30%, low-priority AI | 17.3k, 97.0%, 0.035% | 17.6k, 99.2%, 0.048% | 17.6k, 98.5%, 0.112% |
| Fixed 70%, low-priority AI | 40.1k, 94.1%, 0.054% | 41.1k, 96.6%, 0.051% | 41.2k, 96.3%, 0.117% |
| No AI | 0, 100%, 0.032% | 0, 100%, 0.037% | 0, 100%, 0.090% |

**GPUs.** The rule of Antiphase has no value that depends on the number of neural receivers (Figure 18). With one GPU and one two-user cell, Antiphase serves 8.5k tokens/s at 99.8%, and the 70% share with low-priority AI serves 9.3k at 92.3%. With one GPU and two two-user cells, one neural receiver cannot serve the demand: 31% of the candidates are lost without AI. Antiphase then keeps 97.7% of the recoveries, the level of *Static* and of *Priority*, and serves 1.6-4.5× their AI. With one GPU, Antiphase misses the layer-1 deadline for 0.04-0.06% of the TBs against 0.02-0.04% without AI in these runs of 10 s. The difference comes from the start of the runs: in runs of 100 s no TB is late after the first 20 s with Antiphase or without AI, and *Static* and *Priority* miss the deadline for 46 and 21 of 0.8 million TBs (Table 18).

**Pooling.** With the same demand per GPU, a larger server loses fewer candidates, because the candidates of several cells share several neural receivers (Figure 19). With one two-user cell per GPU, 17.9% of the candidates find no neural receiver on one GPU, 6.9% on two, 4.0% on three, and 1.7% on four, without AI. Antiphase is within 0.3 points of these values at every size. The relative cost of a share grows with the server: *Priority-70* loses 1.4× the candidates of the server without AI on one GPU and 3.6× on four (25.0%, 14.1%, 8.6%, and 6.2%). The part that is lost to a lack of receivers shrinks as the server grows; the part that is lost because AI makes a run hold one more slot does not. The event simulation of §6.3 gives 18.1%, 6.2%, 4.1%, and 1.7% without AI and 1.2% and 1.1% for five and six GPUs, which we did not run.

### 6.11 More cells and other loads

**Takeaway.** With 20, 32, and 48 cells on four GPUs, Antiphase serves 5.9-6.2× the AI of *Static* and 1.8-1.9× that of *Priority*, and it loses at most 0.5% of the recoveries (Figure 18b).

**Cells.** Table 26 keeps four GPUs and adds cells: 20 cells at full load, five per GPU, and 32 and 48 cells where each cell carries a TB in a slot with probability 0.5 and 0.33. Antiphase serves 31.4k-33.4k

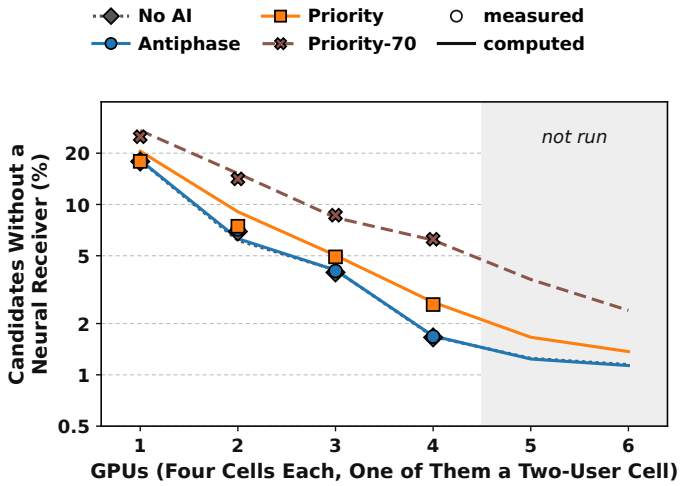



Fig. 19. Candidates that find no neural receiver with one two-user cell per GPU (log scale). Markers: measured on one to four GPUs. Lines: computed; five and six GPUs were not run.

Table 27. Lost candidates with 20 cells for two time bounds of the neural receiver (two seeds). The prediction is the closed form with the arrivals and run lengths measured at 8.6 ms, computed before the runs at 7.6 ms. Antiphase here allows only the smallest unit class next to five active cells.

| Policy | Measured, 8.6 ms | Predicted, 7.6 ms | Measured, 7.6 ms | Three-slot runs, predicted | Three-slot runs, measured |
|---|---|---|---|---|---|
| No AI | 3.6% | 1.6% | 1.6% | 11% | 17% |
| Antiphase | 6.4% | 1.8% | 1.8% | 15% | 18% |
| 30%, low-priority AI | 6.8% | 3.1% | 3.8% | 43% | 47% |
| 70%, low-priority AI | 7.0% | 5.7% | 6.2% | 92% | 92% |

tokens/s and keeps 99.5-100.0% of the recoveries, 5.9-6.2× the AI of the fixed 10% share and 1.8-1.9× that of *Priority*. The 70% share with low-priority AI serves 23-28% more and loses 3.4-5.9% of the recoveries. In these runs of 10 s, Antiphase misses the layer-1 deadline for 0.044% and 0.041% of the TBs with 20 and 32 cells against 0.032% and 0.037% without AI, and with 48 cells every policy with AI misses it more often than the run without AI (0.11-0.13% against 0.09%). Most of these misses come from the start of the runs. In runs of 100 s without their first 20 s, Antiphase has 3 late TBs of 2.3 million with 20 cells and 17 of 1.9 million with 48 cells, against 2 and 17 without AI (Table 18; with the later runs, 93 of 8.5 million against 4, and 39 of 3.8 million against 35), and it adds 0.03 ms to the 99.9th-percentile L1 latency where *Static* and *Priority* add 0.16-0.25 ms. With 48 cells, up to 12 cells of a GPU are active in one slot, and nine TBs per million are late with and without AI.

**Choosing the Time Bound with the Model.** With 20 cells, a time bound of 8.6 ms for the neural receiver gives a latest start time of 2.9 ms and a slack of 0.05 ms in Equation 3. Without AI, 47% of the runs take three slots, and every policy loses 3-5% of the recoveries. Before we ran a bound of 7.6 ms, we computed its loss with the closed form from the arrivals and run lengths measured at 8.6 ms. Table 27 compares the prediction with the runs: it is within 0.7 points for all four policies. With the bound of 7.6 ms, the run without AI recovers 190 TBs per second against 185, and at most 0.2% of the neural receiver runs end after the recovery deadline.

Table 28. Headline setting with a later recovery deadline (three seeds; the first row five): AI served (tokens/s) / recoveries kept.

| Recovery deadline | Antiphase | 30%, low priority | 70%, low priority | Low priority, no cap |
|---|---|---|---|---|
| 11.5 ms (default) | 34.1k / 99.9% | 18.0k / 98.8% | 42.4k / 94.4% | 49.0k / 93.7% |
| 12.0 ms | 33.7k / 99.8% | 17.8k / 99.0% | 41.4k / 95.4% | 48.4k / 93.9% |
| 12.5 ms | 33.9k / 99.8% | 17.6k / 99.6% | 41.1k / 98.0% | 48.0k / 93.9% |

**AI Load.** With 13k and 27k tokens/s offered at a full radio load of 16 cells, Antiphase serves every request and keeps 99.4-99.8% of the recoveries: it rejects 0.0% and 0.6% of the requests, and the 99th percentile of the request latency is 77 and 115 ms. At 27k offered, *Priority* serves 18.9k and rejects 13.2%. The 16% of requests that Antiphase rejects at 53k offered are the load above its capacity. At 106k tokens/s offered, every policy serves less than at 53k: the admission accepts shorter requests, which take more GPU time per token. Antiphase serves 24.3k tokens/s at 100.0%, and *Priority-70* serves 27.9k at 93.6%. When AI requests arrive in bursts (inter-arrival times with a coefficient of variation of 4), Antiphase serves 23.3k tokens/s at 99.5%, *Static* 3.6k at 99.0%, and *Priority-70* 27.4k at 95.9%.

**A Later Recovery Deadline.** Every result so far uses a recovery deadline of 11.5 ms. With a MAC lead of three slots the decision for the retransmission slot could be as late as 12.0-12.5 ms (§3.2), so we ran the headline setting at these two deadlines with the same time limit (Table 28; the latest start becomes 4.4 and 4.9 ms). Antiphase does not change: it serves 33.7k-34.1k tokens/s and keeps 99.8-99.9% of the recoveries at the three deadlines, because it never lengthens a neural receiver run and has nothing to gain from a later deadline. The shares gain: a later latest start turns fewer of their lengthened runs into three-slot runs (86%, 46%, and 21% for the 70% share). At 12.5 ms the 70% share serves 1.2× the AI of Antiphase and keeps 98.0% of the recoveries, ten times the loss of Antiphase, with a layer-1 tail of 3.55 ms against 3.14 ms. Low priority without a cap stays at 93.9%: fewer candidates find no neural receiver (3.7% against 5.7%) and more runs end after the deadline (2.2% against 0.4%). Under rate control the 70% share does not come closer. With eight two-user cells at the 1% target it loses 4.03%, 4.29%, and 4.39% of the goodput at 11.5, 12.0, and 12.5 ms, and with four two-user cells 0.5-0.8% at 11.5 and 12.5 ms; Antiphase loses at most 0.4% in the same settings (0.35% and 0.15% at 12.0 and 12.5 ms with eight two-user cells), the 30% share 1.75% and 1.57%, and low priority without a cap 5.95% and 5.58%. The statement that no cap serves more AI than Antiphase at the same loss of goodput (§6.6) therefore holds at the three deadlines; the 70% share approaches Antiphase only in the recoveries kept at a fixed MCS. We committed the closed-form prediction of the candidates that are not recovered in time before these runs: the ten values have a mean absolute error of 0.59 points; no AI, Antiphase, and the 30% share are within 0.2 points, and the 70% share and no cap are 0.3-2.5 points low, because the runs that end after the deadline were predicted at 0.1% and are 0.2-2.2%. Figure 9 extends the comparison to thirteen deadlines with an event simulation.

**A Longer Replay.** A replay of 128 slots per cell had biased earlier results by 1.0-1.5 points of the recoveries kept. To check the 512 slots used here, we ran the headline setting with a replay of 1,000 slots, all the slots of the two-user cells in the data set (three seeds, 20 s). Antiphase serves 32.9k tokens/s and keeps 99.9% of the recoveries (34.1k and 99.9% with 512 slots), the 30% share 17.5k and 99.1% (18.0k, 98.8%), the 70% share 41.2k and 95.4% (42.4k, 94.4%), and the fixed 10% share 5.4k and 99.4% (5.3k, 99.4%). The factors of Antiphase are 1.88× and 6.1× against 1.89× and 6.4×.

Table 29. Five kinds of AI work on every GPU (16 cells at full load, three seeds, every policy in one job). Chat: output tokens/s on time, and in parentheses the responses with every token on time and the rejected requests. Larger LLM: prompt tokens/s within the limit and rejected requests.

| Policy | Recoveries kept | Chat | Larger LLM | Embedding items/s | Image items/s | Filler tokens/s |
|---|---|---|---|---|---|---|
| Antiphase | 99.7% | 119 (92%, 10%) | 3.7k (11%) | 593 | 237 | 5.4k |
| Fixed 10% | 99.3% | 6 (0%, 64%) | 0.0k (67%) | 117 | 36 | 3.0k |
| Fixed 30%, low-priority AI | 98.5% | 81 (31%, 29%) | 1.2k (28%) | 542 | 210 | 3.9k |
| Fixed 30% | 97.3% | 95 (55%, 25%) | 2.2k (24%) | 572 | 224 | 4.6k |
| Fixed 70%, low-priority AI | 96.5% | 141 (100%, 2%) | 4.5k (2%) | 593 | 237 | 12.8k |
| Low-priority AI, no cap | 94.0% | 154 (100%, 0%) | 4.6k (0%) | 594 | 238 | 18.6k |

### 6.12 Several kinds of AI work

**Takeaway.** With four models and five kinds of AI work on every GPU, the same rule keeps 99.7% of the recoveries. Antiphase serves 1.5× the chat tokens and 3.1× the larger-LLM tokens of *Priority*.

Every GPU runs five kinds of AI work with four models: chat with Qwen2.5-1.5B (prefill, then up to 64 output tokens; first token within 200 ms, every later token within 50 ms; 0.8 requests/s), prefill with Qwen2.5-3B (400 ms; 2 requests/s), text embedding (50 ms; 150 items/s), image classification (50 ms; 60 items/s), and a filler of 1,024-token prefill without a limit. The work with a limit needs 65% of the GPU time. Table 29 shows the result.

Antiphase keeps 99.7% of the recoveries with the rule unchanged. It serves every embedding and image item, 80% of the larger-LLM prompt tokens and 77% of the chat output tokens that *Priority-max* serves, and 92% of its chat responses have every token on time. Against the mechanisms that keep at least 98.5% of the recoveries, Antiphase serves 1.5× the chat output tokens of *Priority*, with 92% against 31% of the responses fully on time, and 3.1× the larger-LLM tokens. Antiphase serves less of the work without a limit: *Priority-70* and *Priority-max* serve 2.4-3.4× the filler tokens, at 3.2-5.7 points fewer recoveries. Antiphase rejects 10% of the chat requests and 11% of the larger-LLM requests, because the AI of a GPU stops for 6.4 ms while its neural receiver runs. With chat alone (1.5 requests/s per GPU, two seeds), Antiphase serves 258 output tokens/s with every response on time and keeps 99.7%, and *Priority-max* serves 300 and keeps 97.2%.

### 6.13 Choosing the TBs to recover

On the urban micro channel, sending every failed TB to the neural receiver takes 4,891-4,926 runs and recovers 663-757 TBs without AI. Sending only the TBs of slots with at most nine failed code blocks takes 651-777 runs and recovers 589-746 TBs: 85% fewer runs for 5% fewer recoveries. On the low-correlation channel with eight two-user cells, the neural receivers cannot take every failed TB. Sending only slots with at most three failed code blocks removes 40% of the runs and 22% of the recoveries and raises the recoveries per run from 0.77 to 0.99.

## 7 Related Work

**GPU sharing in vRAN and AI-RAN.** YinYangRAN [25] multiplexes LDPC decoding with other workloads by MPS shares that a learned reliability estimator changes at intervals of at least one second. CloudRIC [26] pools accelerators for LDPC decoding, and Concordia [13] reserves capacity for vRAN tasks. Weaver [34] shares the GPU of a RAN server between LDPC decoding and the training of foundation models with CUDA green contexts, which give each side a set of streaming multiprocessors (SMs), and it sets the SMs of the radio per slot from the MCS and the PRBs. Its radio work is LDPC decoding without a neural receiver. We build the same means into our code: a neural

receiver that gives up 26% of the SMs runs 15% longer, which is 70% of its slack, and AI that stays on a slice of the SMs loses more goodput than Antiphase (Section 6.6). AI-RAN studies [5, 19, 27, 28] place AI and RAN workloads across servers. Their radio work has one deadline and no second receiver. Antiphase adds a recovery path with its own deadline, and our evaluation includes a share chosen by an estimator and a share that follows the load as baselines.

**GPU scheduling.** DARIS, REEF, XSched, and nvtaskset [6, 7, 15, 29] schedule GPU stages, kernels, or contexts by priority, Orion [30] runs best-effort GPU work when it does not interfere with a high-priority job, and GPreempt [12] preempts a GPU context within 40 $\mu$s. SMEC [35] sets MPS priorities on an edge server that is separate from the RAN server. These systems protect the latency of the running high-priority job. Our model shows that this is not the quantity that decides the recoveries: a neural receiver that meets its deadline still costs recoveries when it holds its GPU for one more slot. Antiphase decides from the state of every neural receiver, and our evaluation includes low priority as a baseline.

**Neural receivers.** DeepRx [17] and the NVlabs receivers [9, 32] design and train neural receivers, ARCHES [24] switches between an AI channel estimator and a conventional one, and OCUDU [22] exposes timing classes in the distributed unit. These works report receiver gains against their own baselines. We compare at equal LDPC effort with a production receiver and study the neural receiver as a workload that shares the GPU with AI. Nuberu [14] keeps the HARQ timing of a virtualized DU under a compute shortage by inferring whether a TB will decode before the decode ends. Antiphase infers nothing: it uses the asynchronous uplink HARQ of NR and withholds the retransmission grant until the result of the neural receiver is in.

**Loss models.** Loss systems describe servers without a waiting room [16]. Our model is a loss system in which the service time counts in uplink slots, the arrivals of consecutive slots are dependent, and the policy enters through the distribution of the run length.

## 8 Limitations

The channels are the synthetic channels of the public NVlabs configuration, and every cell replays 512 slots (a replay of 1,000 slots gives the same results in the headline setting). Antiphase serves 76-95% of the AI of a clairvoyant controller that was run and 43-88% of an estimate for an offline schedule; it does not predict which neural receiver will be needed; a predictor would have to find nine of ten candidates to keep 99% of the recoveries, and exact knowledge is worth 15% more AI. The clairvoyant controller is one rule and not the optimum. At a recovery deadline of 12.5 ms and a fixed MCS, a low-priority share with a cap of 70% keeps 98.0% of the recoveries and serves 1.2× the AI of Antiphase; under rate control it loses 0.5-4.4% of the goodput at every deadline we ran. The rule on unit sizes costs 9-12% of the AI served, and we measure no effect of it on the late TBs in steady state. The larger model allows four to five cells per GPU at full load; 32 and 48 cells run when half or two thirds of the cells are idle, and with 48 cells nine TBs per million miss the layer-1 deadline in steady state with and without AI. The recovery deadline and the retransmission slot follow from the TDD pattern, the PUSCH preparation time [4], and a MAC lead of three slots; they are not validated with a layer-2 stack, and the path that hands the MAC a late CRC pass of a TB already reported as failed is not built; the goodput results apply a retransmission schedule to measured TB records and do not run HARQ; the assumed success of a retransmission does not change the comparison of the policies under rate control. The outer loop runs on three channels at seven Es/No values, with an Es/No that changes in steps and continuously, and with two channels on one server; the continuous change adds noise to generated slots, and the channel itself is replayed. On the urban micro channel the outer loop stays at the lowest MCS at the 3% target. The goodput that the shares with more AI give up is 0.3-0.6 percentage points at a fixed MCS and at most 2.5% with the outer loop at a 10% target; it reaches 8.2% at a 1% target, and it is at

most 0.9% on the low-correlation channel and 0.7% on the urban micro channel. The closed form takes the run lengths of the neural receiver from runs of the policy; it does not predict them from the policy. The baselines are the mechanisms of prior systems on our code: the estimator follows the published design, with our reliability definition and without the look-ahead of the full system. Antiphase raises the median L1 latency by 0.4 ms. Antiphase serves less AI without a time limit than *Priority-max*. The layer-1 misses of 0.02-0.13% in the runs of 10-40 s come from the cold start of the prototype, with and without AI. After the start of a run Antiphase has four to eight late TBs per million against two without AI. The steady-state counts are from 1.3-8.5 million TBs per setting and do not establish a reliability beyond 99.999%; the changing loads and 32 cells are not measured in runs of 100 s. The SM slices have a fixed size, two seeds, and one AI model, and we build them with the driver API because the MPS of this server cannot divide the SMs; we do not run Weaver. All runs use one GPU model.

## 9 Conclusion

A neural receiver that decodes more TBs than a production receiver at equal LDPC effort takes twice the uplink period. Running it as a recovery path next to AI inference loses recoveries by a path that deadline checks miss: a failed TB cannot wait, and a neural receiver that AI slows by more than the slack of 1.2 ms holds its GPU for one more uplink slot. A loss model of this path agrees with 672 runs within 0.82 percentage points on average and predicted 24 values of six settings, before they were run, within 0.34 points. Antiphase stops the low-priority AI of a GPU when its neural receiver starts and keeps the largest AI units away from the conventional receivers. Under three load patterns with mixed channels, Antiphase serves 6.4-7.5× the AI of a fixed 10% GPU share and 1.2-1.9× that of a share chosen by a load estimator with low-priority AI, and it keeps 99.9% of the recoveries. In steady state it adds 0.04-0.06 ms to the 99.9th-percentile latency of the layer-1 decode, and after the start of a run four to eight of a million TBs pass the layer-1 deadline, against two without AI. It serves 87% of the AI of a clairvoyant controller that we ran, 1.9-6.4× the existing mechanisms that keep the recoveries. With an outer-loop rate controller, lost recoveries lower the MCS: the cost in goodput reaches 2.5% at a 10% retransmission target and 8% at a 1% target with a neural receiver demand near the capacity. Antiphase loses at most 0.5% of the goodput in 42 settings with one configuration, which include three channels and a continuously changing SNR, and no fixed or low-priority GPU share does.

## References


[1] 3GPP. 2020. *Study on Channel Model for Frequencies from 0.5 to 100 GHz.* 3rd Generation Partnership Project. https://portal.3gpp.org/desktopmodules/Specifications/SpecificationDetails.aspx?specificationId=3173 Technical Report 38.901.

[2] 3GPP. 2024. *NR; Medium Access Control (MAC) Protocol Specification.* 3rd Generation Partnership Project. https://portal.3gpp.org/desktopmodules/Specifications/SpecificationDetails.aspx?specificationId=3194 Technical Specification 38.321, clause 5.4.2.

[3] 3GPP. 2024. *NR; NR and NG-RAN Overall Description; Stage-2.* 3rd Generation Partnership Project. https://portal.3gpp.org/desktopmodules/Specifications/SpecificationDetails.aspx?specificationId=3191 Technical Specification 38.300, clause 5.3.5.4.

[4] 3GPP. 2024. *NR; Physical Layer Procedures for Data.* 3rd Generation Partnership Project. https://portal.3gpp.org/desktopmodules/Specifications/SpecificationDetails.aspx?specificationId=3216 Technical Specification 38.214.

[5] Ganesh Ananthanarayanan, Matthew Balkwill, Xenofon Foukas, Zhihua Lai, Bozidar Radunovic, Connor Settle, and Yongguang Zhang. 2025. Distributed AI Platform for the 6G RAN. In *ACM Workshop on Open and AI RAN.* ACM, New York, NY, USA, 15–21. https://doi.org/10.1145/3737900.3770167

[6] Amir Fakhim Babaei and Thidapat Chantem. 2025. DARIS: An Oversubscribed Spatio-Temporal Scheduler for Real-Time DNN Inference on GPUs. In *ACM/IEEE Design Automation Conference.* IEEE, Piscataway, NJ, USA, 1–7. https://doi.org/10.1109/DAC63849.2025.11132423

[7] Joshua Bakita and James H. Anderson. 2025. Hardware Compute Partitioning on NVIDIA GPUs for Composable Systems. In *ECRTS*, Vol. 335. Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, Wadern, Germany, 21:1–21:25. https://doi.org/10.4230/LIPIcs.ECRTS.2025.21

[8] Francisco Blanquez-Casado, Gerardo Gomez, Maria del Carmen Aguayo-Torres, and Jose Tomas Entrambasaguas. 2016. eOLLA: An Enhanced Outer Loop Link Adaptation for Cellular Networks. *EURASIP Journal on Wireless Communications and Networking* 2016, 1 (2016), 20. https://doi.org/10.1186/s13638-016-0518-3

[9] Sebastian Cammerer, Fayçal Ait Aoudia, Jakob Hoydis, Andreas Oeldemann, Andreas Roessler, Timo Mayer, and Alexander Keller. 2023. A Neural Receiver for 5G NR Multi-User MIMO. In *IEEE Globecom Workshops*. IEEE, Piscataway, NJ, USA, 329–334.

[10] Sebastian Cammerer, Reinhard Wiesmayr, Fayçal Ait Aoudia, Jakob Hoydis, Tommi Koivisto, Jakub Zakrzewski, Ruqing Xu, Pawel Morkisz, Chris Dick, and Alexander Keller. 2024. Real-time 5G NR Multi-user MIMO Receivers. https://github.com/NVlabs/neural-rx.

[11] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. 2020. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. https://doi.org/10.48550/arXiv.2010.11929 arXiv:2010.11929

[12] Ruwen Fan, Tingxu Ren, Minhui Xie, Shiwei Gao, Jiwu Shu, and Youyou Lu. 2025. GPREEMPT: GPU Preemptive Scheduling Made General and Efficient. In *2025 USENIX Annual Technical Conference (USENIX ATC 25)*. 263–272. https://www.usenix.org/conference/atc25/presentation/fan

[13] Xenofon Foukas and Bozidar Radunovic. 2021. Concordia: Teaching the 5G vRAN to Share Compute. In *ACM SIGCOMM*. ACM, New York, NY, USA, 580–596. https://doi.org/10.1145/3452296.3472894

[14] Gines Garcia-Aviles, Andres Garcia-Saavedra, Marco Gramaglia, Xavier Costa-Perez, Pablo Serrano, and Albert Banchs. 2021. Nuberu: Reliable RAN Virtualization in Shared Platforms. In *Proceedings of the 27th Annual International Conference on Mobile Computing and Networking (MobiCom)*. 749–761. https://doi.org/10.1145/3447993.3483266

[15] Mingcong Han, Hanze Zhang, Rong Chen, and Haibo Chen. 2022. Microsecond-Scale Preemption for Concurrent GPU-Accelerated DNN Inferences. In *USENIX OSDI*. USENIX Association, Boston, MA, USA, 539–558. https://www.usenix.org/conference/osdi22/presentation/han

[16] Mor Harchol-Balter. 2013. *Performance Modeling and Design of Computer Systems: Queueing Theory in Action*. Cambridge University Press, Cambridge, UK. https://doi.org/10.1017/CBO9781139226424

[17] Mikko Honkala, Dani Korpi, and Janne M. J. Huttunen. 2021. DeepRx: Fully Convolutional Deep Learning Receiver. *IEEE Transactions on Wireless Communications* 20, 6 (2021), 3925–3940. https://doi.org/10.1109/TWC.2021.3054520

[18] Jakob Hoydis, Sebastian Cammerer, Fayçal Ait Aoudia, Avinash Vem, Nikolaus Binder, Guillermo Marcus, and Alexander Keller. 2022. Sionna: An Open-Source Library for Next-Generation Physical Layer Research. arXiv:2203.11854 https://arxiv.org/abs/2203.11854

[19] Haiyuan Li, Yulei Wu, and Dimitra Simeonidou. 2026. Deadline-Driven Hierarchical Agentic Resource Sharing for AI Services and RAN Functions in AI-RAN. arXiv:2605.07547 [cs.NI] https://arxiv.org/abs/2605.07547

[20] NVIDIA Corporation. 2026. *Aerial CUDA-Accelerated RAN Documentation*. NVIDIA. https://docs.nvidia.com/aerial/cuda-accelerated-ran/

[21] NVIDIA Corporation. 2026. *Multi-Process Service Documentation*. NVIDIA. https://docs.nvidia.com/deploy/mps/

[22] Timothy O'Shea, Matthew Pennybacker, and Andriy Kharchenko. 2026. The OCUDU dApp Platform: An Open Runtime and E3 Interface for Real-Time AI-RAN. arXiv:2609.07843 [cs.NI] https://arxiv.org/abs/2609.07843

[23] Qwen Team. 2024. Qwen2.5 Technical Report. arXiv:2412.15115 https://arxiv.org/abs/2412.15115

[24] Neagin Neasamoni Santhi, Davide Villa, Michele Polese, Salvatore D'Oro, Yunseong Lee, Koichiro Furueda, and Tommaso Melodia. 2026. ARCHES: Adaptive Real-Time Switching of AI Models for the RAN. arXiv:2604.23397 [cs.NI] https://arxiv.org/abs/2604.23397

[25] Leonardo Lo Schiavo, Jose A. Ayala-Romero, Andres Garcia-Saavedra, Marco Fiore, and Xavier Costa-Perez. 2024. YinYangRAN: Resource Multiplexing in GPU-Accelerated Virtualized RANs. In *IEEE INFOCOM*. IEEE, Piscataway, NJ, USA, 721–730. https://doi.org/10.1109/INFOCOM52122.2024.10621380

[26] Leonardo Lo Schiavo, Gines Garcia-Aviles, Andres Garcia-Saavedra, Marco Gramaglia, Marco Fiore, Albert Banchs, and Xavier Costa-Perez. 2024. CloudRIC: Open Radio Access Network (O-RAN) Virtualization with Shared Heterogeneous Computing. In *ACM MobiCom*. ACM, New York, NY, USA, 558–572. https://doi.org/10.1145/3636534.3649381

[27] Syed Danial Ali Shah, Maryam Hafeez, Abdelaziz Salama, and Syed Ali Raza Zaidi. 2025. Proactive AI-and-RAN Workload Orchestration in O-RAN Architectures for 6G Networks. arXiv:2507.09124 [cs.NI] https://arxiv.org/abs/2507.09124

[28] Syed Danial Ali Shah, Zeinab Nezami, Maryam Hafeez, and Syed Ali Raza Zaidi. 2025. The Interplay of AI-and-RAN: Dynamic Resource Allocation for Converged 6G Platform. In *IEEE INFOCOM Workshops*. IEEE, Piscataway, NJ, USA, 1–6. https://doi.org/10.1109/INFOCOMWKSHPS65812.2025.11152939

[29] Weihang Shen, Mingcong Han, Jialong Liu, Rong Chen, and Haibo Chen. 2025. XSched: Preemptive Scheduling for Diverse XPUs. In *USENIX OSDI*. USENIX Association, Boston, MA, USA, 671–692. https://www.usenix.org/conference/osdi25/presentation/shen-weihang

[30] Foteini Strati, Xianzhe Ma, and Ana Klimovic. 2024. Orion: Interference-aware, Fine-grained GPU Sharing for ML Applications. In *ACM EuroSys*. ACM, Athens, Greece, 1075–1092. https://doi.org/10.1145/3627703.3629578

[31] Yuxin Wang, Yuhan Chen, Zeyu Li, Xueze Kang, Yuchu Fang, Yeju Zhou, Yang Zheng, Zhenheng Tang, Xin He, Rui Guo, Xin Wang, Qiang Wang, Amelie Chi Zhou, and Xiaowen Chu. 2025. BurstGPT: A Real-World Workload Dataset to Optimize LLM Serving Systems. In *ACM SIGKDD*. ACM, New York, NY, USA, 5831–5841. https://doi.org/10.1145/3711896.3737413

[32] Reinhard Wiesmayr, Sebastian Cammerer, Fayçal Ait Aoudia, Jakob Hoydis, Jakub Zakrzewski, and Alexander Keller. 2024. Design of a Standard-Compliant Real-Time Neural Receiver for 5G NR. arXiv:2409.02912 https://arxiv.org/abs/2409.02912

[33] Shitao Xiao, Zheng Liu, Peitian Zhang, Niklas Muennighoff, Defu Lian, and Jian-Yun Nie. 2024. C-Pack: Packed Resources For General Chinese Embeddings. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*. ACM, New York, NY, USA, 641–649. https://doi.org/10.1145/3626772.3657878

[34] Leyang Xue, Tianxin Wang, Xin Zhe Khooi, Jiaxun Yang, Dheeraj Mahendiran, Yufeng Xia, Mun Choon Chan, Myungjin Lee, and Mahesh K. Marina. 2026. Weaver: A System for AI-RAN Compute Sharing with Foundation Model Training. arXiv:2609.35276 [cs.DC] https://arxiv.org/abs/2609.35276

[35] Xiao Zhang and Daehyeok Kim. 2026. Enabling SLO-Aware 5G Multi-Access Edge Computing with SMEC. In *USENIX NSDI*. USENIX Association, Boston, MA, USA, 1759–1776. https://www.usenix.org/conference/nsdi26/presentation/zhang-xiao